

Alcance de la tecnología: la infraestructura como código

Funcionamiento de un sistema completo, capas y módulos principales, matriz de estructura de dependencia, modelo de objetos y procesos, y posicionamiento frente a la competencia

Carlos Luis Rojas Aragonés
Gestión del Desarrollo Tecnológico

1 de septiembre de 2026

Resumen

La infraestructura como código (*Infrastructure as Code*, IaC) sustituye la configuración manual de servidores, redes y servicios por una especificación versionada que un motor automático traduce en operaciones sobre la interfaz de programación de un proveedor. Este trabajo delimita el alcance de la tecnología, describe su principio de funcionamiento (la conciliación continua entre un estado deseado y un estado real), desglosa las siete capas y los seis módulos transversales de un sistema completo, y ordena sus interdependencias en dos matrices de estructura de dependencia, una de arquitectura y otra de cartera de proyectos, cuyo particionado produce la secuencia de implantación. Formaliza después el conjunto mediante un modelo de objetos y procesos según la metodología OPM, con su diagrama de sistema, el despliegue estructural de la especificación, la ampliación en detalle del proceso principal y el lenguaje OPL de los tres diagramas. Sobre las figuras de mérito de la tecnología se construye por último el posicionamiento de la empresa frente a la competencia, con un gráfico vectorial que sitúa a Terraform, Bicep y Pulumi en un plano de variación de valor para el cliente y variación de coste para el productor, y que descompone la trayectoria de cada jugador en los vectores tecnológicos que la producen. Todas las figuras son de elaboración propia.

Índice

1. Objeto y delimitación del alcance	4
01. El encargo	4
02. Qué se entiende aquí por sistema completo	4
03. La frontera del sistema	5
2. El principio de funcionamiento	6
01. Tres estados y una función	6
02. Declarativo frente a imperativo	6
03. Convergencia y deriva	7
3. El ciclo básico de operación	7
01. El paso crítico: la comparación a tres bandas	8
4. Las capas de un sistema completo	8
01. Qué hace cada capa	8
02. La tensión entre capas	10
5. Los módulos transversales	10
6. Modos de operación	10
01. Empuje frente a arrastre	11
02. Mutable frente a inmutable	11
03. El cuadro de decisión	12
7. El módulo como unidad de composición	12
8. Matriz de estructura de dependencia (DSM)	13
01. Convención de lectura	14
02. La DSM de arquitectura	14
03. Qué dice la matriz sin reordenarla	15
04. Particionado: la secuencia de implantación	16
05. La DSM de la cartera de proyectos	17
06. Secuenciación de la cartera	17
07. Riesgos que la matriz hace visibles	18
08. Límites de las dos matrices	19
9. Modelo de objetos y procesos (OPM)	20
01. Por qué OPM y cómo se ha construido el modelo	20
02. Vocabulario y notación empleados	20
03. Diagrama de sistema (SD)	21
04. El OPL del diagrama de sistema	22
05. Despliegue de la especificación (SD2)	23
06. Ampliación en detalle del proceso principal (SD1)	24
07. El OPL de la ampliación	24
08. Qué hace explícito el modelo	25
10. Figuras de mérito del sistema	25
11. Posicionamiento frente a la competencia	26
01. Qué se compara y desde qué punto de vista	26
02. Los tres competidores considerados	27

03. Construcción de las dos coordenadas	27
04. Estimación de las figuras de mérito al horizonte 2029	28
05. Descomposición en vectores tecnológicos	29
06. El gráfico vectorial	29
07. Lectura del gráfico	30
08. Sensibilidad al supuesto de cartera	31
09. Límites del análisis	32
12. Conclusiones	33
Referencias bibliográficas	34

1. Objeto y delimitación del alcance

01. El encargo

El documento recoge entregas encadenadas de una misma hoja de ruta tecnológica, y por eso no se organiza por actividades sino por los elementos del índice recomendado para una hoja de ruta. El Cuadro 1 recoge esa correspondencia.

La primera entrega, el alcance de la tecnología, cubre los tres primeros elementos. El elemento 1, la descripción de la tecnología, ocupa las secciones 2 a 7, con el resumen funcional y los diagramas de capas y de módulos. El elemento 2, la matriz de estructura de dependencia, ocupa la sección 8 con dos matrices, una de los componentes del sistema y otra de los proyectos de la cartera, y con el particionado que convierte a ambas en una secuencia de implantación. El elemento 3, el modelo de objetos y procesos, ocupa la sección 9, con el diagrama de sistema, el despliegue estructural de la especificación, la ampliación en detalle del proceso principal y el OPL de los tres diagramas.

La segunda entrega pide analizar la competencia mediante un gráfico vectorial que incluya al menos dos competidores. Corresponde al elemento 6 y ocupa la sección 11, que compara tres jugadores, Terraform, Bicep y Pulum, con la trayectoria propuesta para la empresa. Su sitio natural es inmediatamente después de la sección 10, porque las coordenadas del gráfico se construyen a partir de las figuras de mérito definidas allí.

Cuadro 1: Correspondencia entre los doce elementos del índice recomendado para una hoja de ruta tecnológica y las secciones de este documento.

N.º	Elemento	Dónde se atiende
1	Descripción de la tecnología	Secciones 1 a 7
2	Matriz de estructura de dependencia (DSM)	Sección 8
3	Modelo con metodología OPM	Sección 9
4	Figuras de mérito (FOM)	Sección 10
5	Correspondencia con las motivaciones estratégicas	Pendiente
6	Posicionamiento frente a la competencia	Sección 11
7	Modelo técnico	Pendiente
8	Modelo financiero	Declarado fuera de alcance en la sección 11
9	Portfolio de proyectos y prototipos de I+D	Cuadro 6, en lo relativo a su secuencia
10	Publicaciones, presentaciones y patentes clave	Pendiente
11	Declaración de estrategia tecnológica	Pendiente
12	Evaluación del nivel de madurez de la hoja de ruta	Pendiente

02. Qué se entiende aquí por sistema completo

Conviene fijar el término antes de dibujar nada, porque «infraestructura como código» se usa con dos alcances muy distintos. En el sentido estrecho designa una herramienta: un fichero declarativo y un binario que lo aplica. En el sentido amplio, que es el que aquí interesa, designa un sistema sociotécnico completo, con su cadena de herramientas, su modelo de estado, sus controles de gobierno y el equipo que lo opera.

Un sistema completo de IaC es aquel que satisface cuatro propiedades simultáneas (Morris, 2021):

1. **Especificación externalizada.** La configuración deseada de la infraestructura vive fuera de la infraestructura, en artefactos de texto legibles y versionados, no en la memoria de un operador ni en el estado acumulado de las máquinas.
2. **Reproducibilidad.** Aplicar la misma especificación sobre un sustrato vacío produce un resultado equivalente, con independencia de quién la aplique y de cuándo lo haga.
3. **Idempotencia.** Aplicar la especificación sobre un sistema que ya la cumple no produce ningún cambio, y aplicarla n veces tiene el mismo efecto que aplicarla una vez.
4. **Conciliación.** El sistema es capaz de detectar la divergencia entre lo especificado y lo realmente desplegado, y de corregirla.

La literatura académica sobre la materia, todavía escasa en comparación con su difusión industrial, se ha concentrado sobre todo en la calidad y los defectos del propio código de infraestructura (Rahman, Mahdavi-Hezaveh & Williams, 2019).

Un montaje que cumpla sólo la primera propiedad es un repositorio de guiones, no un sistema de IaC. La cuarta es la que convierte la automatización en control, y es también la que con más frecuencia falta en las implantaciones reales.

03. La frontera del sistema

Delimitar el alcance exige decir también qué queda fuera. El Cuadro 2 fija la frontera adoptada en este documento.

Cuadro 2: Frontera del sistema de infraestructura como código.

Dentro del alcance	Fuera del alcance
Cómputo, red, almacenamiento, identidad y servicios gestionados declarados como código.	El sustrato físico del proveedor: centros de datos, cableado, refrigeración.
Configuración del sistema operativo y de los agentes instalados en él.	El código de la aplicación de negocio y sus pruebas funcionales.
Construcción de imágenes y artefactos inmutables.	Los datos de producción y su ciclo de vida, que exigen migraciones y copias de seguridad con reglas propias.
Orquestación de cargas y despliegue continuo de manifiestos.	La gestión de incidencias, la guardia y los procedimientos de crisis.
Estado, secretos, políticas, pruebas y observabilidad de la propia infraestructura.	La observabilidad funcional de la aplicación (trazas de negocio, experiencia de usuario).

La distinción entre infraestructura y datos merece un comentario, porque es la frontera que más problemas causa en la práctica. Un servidor de base de datos está dentro del alcance: su tamaño, su versión, su red y sus copias programadas se declaran como código. El contenido de esa base de datos no lo está: no es idempotente, no se puede destruir y recrear, y su presencia obliga a que el sistema de IaC distinga recursos desechables de recursos con estado persistente, y proteja a los segundos de las operaciones de reemplazo.

2. El principio de funcionamiento

01. Tres estados y una función

Todo el funcionamiento de un sistema de IaC se sostiene sobre tres representaciones de la misma infraestructura y sobre la función que las compara.

Estado deseado.

Lo que la especificación declara. Es un artefacto de texto, versionado, que describe recursos y sus propiedades. No describe los pasos para llegar a ellos.

Estado registrado.

Lo que el motor cree que existe, resultado de la última operación que él mismo ejecutó. Se materializa en un fichero o servicio de estado que asocia cada recurso declarado con el identificador real que el proveedor le asignó (HashiCorp, s.f.-c).

Estado real.

Lo que efectivamente existe en el proveedor, que sólo se conoce consultando su interfaz de programación.

La función central del sistema es la **conciliación**: calcular la diferencia entre el estado deseado y el estado real, y emitir el conjunto mínimo de operaciones que anula esa diferencia. La Figura 1 representa el bucle.

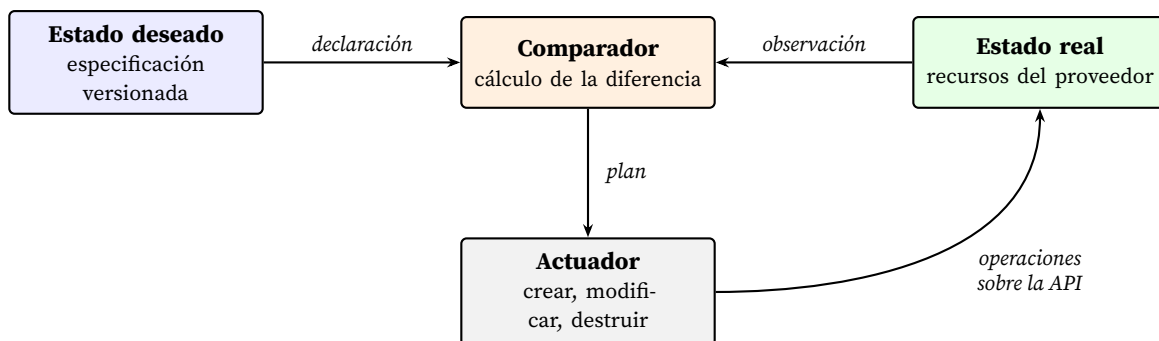


Figura 1: Bucle de conciliación. El sistema no ejecuta una secuencia de pasos, sino que cierra repetidamente la distancia entre dos representaciones. Si la diferencia es vacía, el actuador no ejecuta nada, y ésta es exactamente la propiedad de idempotencia. Elaboración propia.

02. Declarativo frente a imperativo

La diferencia entre los dos paradigmas no es de sintaxis sino de quién calcula el camino. En el enfoque imperativo, el autor escribe la secuencia de operaciones y asume la responsabilidad de que sea correcta desde cualquier estado de partida. En el enfoque declarativo, el autor escribe el destino y el motor deriva la secuencia.

La consecuencia práctica es que un guion imperativo sólo es correcto para el estado de partida que su autor imaginó, mientras que una especificación declarativa es correcta para cualquiera, a condición de que el motor sepa observar el estado real. Por eso el paradigma declarativo domina la capa de aprovisionamiento, donde el espacio de estados de partida es enorme, y sobrevive el imperativo en tareas de configuración muy locales, donde ese espacio es pequeño y conocido.

Casi ningún sistema es puro. Las herramientas de configuración expresan los pasos en orden, pero cada paso es un módulo idempotente que sólo actúa si hace falta (Red Hat, s.f.). Es un híbrido: secuencia imperativa de operaciones declarativas.

03. Convergencia y deriva

Un sistema convergente es el que, aplicado repetidamente, se aproxima al estado deseado y luego se queda quieto. La **deriva** (*drift*) es la aparición de diferencias entre el estado real y el deseado por causas ajenas al sistema: un cambio manual de urgencia, una acción automática del proveedor, la caducidad de un certificado, el borrado accidental de un recurso.

La deriva es inevitable, y la madurez de una implantación se mide por lo que hace con ella. Hay tres posturas, en orden creciente de rigor:

1. **Ignorarla.** El motor sólo se ejecuta cuando alguien lo lanza. Entre ejecuciones, el estado real puede alejarse arbitrariamente.
2. **Detectarla.** Una ejecución periódica en modo de sólo lectura compara y notifica, sin corregir. Convierte la deriva en una métrica observable.
3. **Corregirla.** Un agente concilia de forma continua y revierte todo cambio no declarado. Es el principio de conciliación continua de GitOps (OpenGitOps, s.f.) y el modo nativo de los controladores de Kubernetes (Burns et al., 2016; The Kubernetes Authors, s.f.).

3. El ciclo básico de operación

Descendiendo un nivel desde el bucle abstracto, la operación cotidiana de un sistema completo recorre siete pasos. La Figura 2 los ordena.

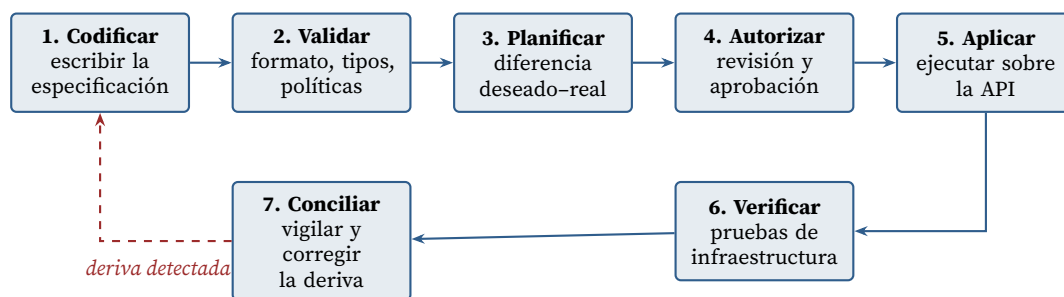


Figura 2: Los siete pasos del ciclo de operación. El retorno discontinuo es el que distingue un sistema de IaC de una simple automatización de despliegue. Elaboración propia.

Los pasos 2, 4 y 6 son los que suelen faltar en implantaciones incompletas, y son precisamente los que aportan garantía. Sin validación de políticas, el sistema automatiza también los errores de configuración; sin autorización, elimina el punto de control humano sobre cambios irreversibles; sin verificación, no distingue entre «el motor terminó sin error» y «la infraestructura funciona».

01. El paso crítico: la comparación a tres bandas

El paso 3 merece detalle porque en él se concentra el mecanismo que hace útil al sistema. El motor no compara dos cosas sino tres, y lo hace en dos fases (Figura 3).

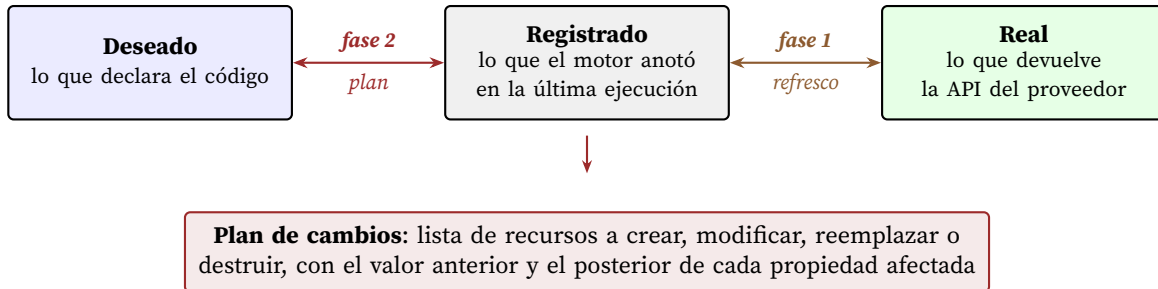


Figura 3: Comparación a tres bandas. La fase de refresco actualiza el registro contra el mundo real y es la que descubre la deriva; la fase de plan compara el código contra ese registro ya actualizado y es la que produce las operaciones. Elaboración propia.

El estado registrado no es redundante. Cumple tres funciones que ninguna de las otras dos representaciones puede cumplir. Primera, traduce el nombre lógico que el código da a un recurso al identificador opaco que el proveedor le asignó, sin lo cual el motor no podría reconocer que un recurso existente le pertenece. Segunda, permite detectar destrucciones: un recurso presente en el registro y ausente del proveedor sólo puede detectarse comparando contra el registro, nunca comparando el código con la realidad. Tercera, guarda propiedades calculadas por el proveedor que el código no declara y que otros recursos necesitan.

Ese fichero es, en consecuencia, un activo crítico: contiene identificadores y con frecuencia valores sensibles, y su corrupción o pérdida deja al sistema sin capacidad de reconocer lo que él mismo creó. De ahí que todo montaje serio lo almacene en un servicio remoto con cifrado, versionado y bloqueo de concurrencia.

4. Las capas de un sistema completo

Un sistema completo se organiza en siete capas apiladas, cada una de las cuales consume el resultado de la anterior y expone una abstracción más alta. La Figura 4 las presenta con sus herramientas representativas.

01. Qué hace cada capa

L0, sustrato. No es parte del sistema de IaC sino su objeto. Aporta la interfaz de programación sobre la que todo lo demás actúa. Su característica determinante es que ofrece recursos con identidad propia, creables y destruibles por llamada, lo que es la condición de posibilidad de todo el resto.

L1, aprovisionamiento. Traduce la especificación en llamadas de creación, modificación y destrucción. Es la capa que posee el registro de estado y la que ejecuta la comparación a tres bandas. Trabaja con recursos que todavía no tienen sistema operativo arrancado, o cuyo interior no le concierne. Conviene señalar que en 2023 el cambio de licencia de Terraform dio origen a OpenTofu,

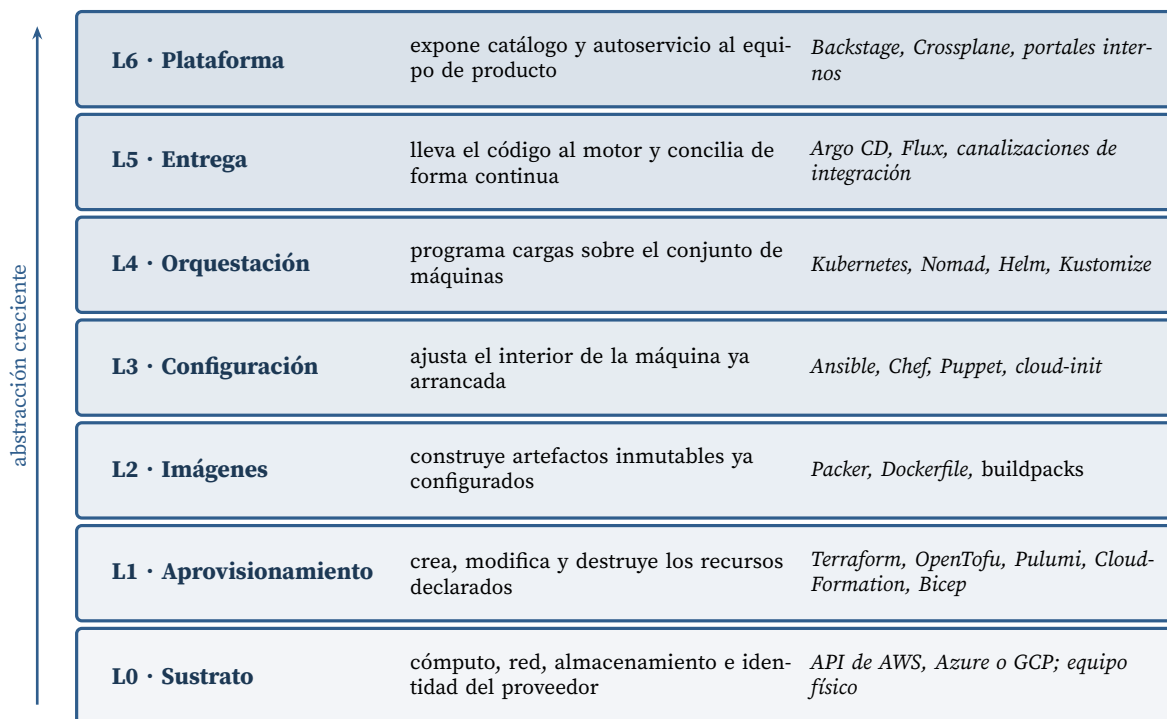


Figura 4: Las siete capas de un sistema completo de infraestructura como código. Cada capa consume la salida de la inferior. Las herramientas citadas son representativas, no exhaustivas. Elaboración propia.

una bifurcación acogida por la Linux Foundation (The Linux Foundation, 2023), de modo que hoy la capa cuenta con dos implementaciones compatibles del mismo lenguaje: una decisión de dependencia que una hoja de ruta tecnológica debería registrar de forma explícita.

L2, imágenes. Desplaza el trabajo de configuración desde el momento de ejecución al momento de construcción. En lugar de arrancar una máquina genérica y modificarla, construye una imagen ya completa y arranca instancias idénticas de ella. Es la base de la infraestructura inmutable: en lugar de modificar un servidor, se reemplaza. Reduce drásticamente la deriva, porque elimina la ventana temporal en la que un servidor puede divergir.

L3, configuración. Actúa sobre el interior de máquinas ya arrancadas: paquetes, ficheros, servicios, usuarios. Es la capa históricamente más antigua y la que más terreno ha cedido a L2 y L4. Sigue siendo necesaria donde hay máquinas de larga vida, sistemas heredados o equipos físicos.

L4, orquestación. Introduce una indirección importante: deja de declararse «esta carga en esta máquina» y pasa a declararse «esta carga, tantas réplicas» sobre un conjunto de máquinas intercambiables. El planificador decide la colocación. Sus controladores son bucles de conciliación como el de la Figura 1, pero embebidos y permanentes (The Kubernetes Authors, s.f.).

L5, entrega. Conecta el repositorio con el motor. Ejecuta las canalizaciones que validan, planifican y aplican, y, en el modelo de conciliación continua, aloja el agente que vigila la deriva sin intervención humana (Humble & Farley, 2010; OpenGitOps, s.f.).

L6, plataforma. Envuelve todo lo anterior en una interfaz de autoservicio, para que un equipo de producto pueda solicitar una base de datos sin conocer ni la capa L1 ni las políticas de la

organización, que quedan codificadas en el catálogo.

02. La tensión entre capas

Las capas L2, L3 y L4 compiten parcialmente por el mismo territorio. Un mismo requisito, por ejemplo «que este servidor tenga la versión 1.24 de un tiempo de ejecución», puede resolverse construyendo una imagen que ya la incluya (L2), instalándola con un módulo de configuración (L3) o empaquetando la aplicación en un contenedor que la lleve dentro (L4). Las tres soluciones funcionan y no deben coexistir sobre el mismo recurso: si dos capas gobiernan la misma propiedad, cada una revierte el trabajo de la otra en cada ciclo de conciliación, y el sistema oscila.

Decidir qué capa posee cada propiedad es, por tanto, la primera decisión de arquitectura de una implantación, y conviene documentarla de forma explícita.

5. Los módulos transversales

Las siete capas describen el flujo principal, pero un sistema completo necesita además seis módulos que atraviesan todas las capas (Figura 5). Ninguno produce infraestructura; todos condicionan si la que se produce es correcta, segura y auditable.

El Cuadro 3 resume la función de cada uno y el fallo característico que aparece cuando falta.

Cuadro 3: Módulos transversales: función, instrumentos y fallo característico por ausencia.

Módulo	Función	Fallo por ausencia
Control de versiones	Registrar quién cambió qué, cuándo y por qué; permitir revertir a un estado anterior conocido.	Cambios sin trazabilidad y reversión imposible.
Gestión de estado	Custodiar el registro: almacenamiento remoto, cifrado, versionado y bloqueo para impedir ejecuciones concurrentes.	Corrupción del registro y recursos huérfanos.
Gestión de secretos	Mantener credenciales fuera del código e inyectarlas en el momento de la ejecución, con rotación y ámbito mínimo. Es uno de los defectos más frecuentes en código de infraestructura (Rahman, Parnin & Williams, 2019).	Credenciales escritas en el repositorio.
Política como código	Expresar las reglas de la organización como predicados evaluables sobre el plan, antes de aplicarlo (Open Policy Agent, s.f.).	El sistema automatiza también los incumplimientos, y a mayor velocidad.
Pruebas	Verificar la especificación en varios niveles: análisis estático, validación del plan, despliegue efímero y comprobación funcional.	«Terminó sin error» se confunde con «funciona».
Observabilidad	Medir deriva, coste, tiempo de ciclo y cumplimiento, y exponerlos como series temporales.	No hay figuras de mérito y la mejora no es demostrable.

6. Modos de operación

Sobre la misma arquitectura caben decisiones de operación que cambian de forma apreciable el comportamiento del sistema. Las tres principales son independientes entre sí.

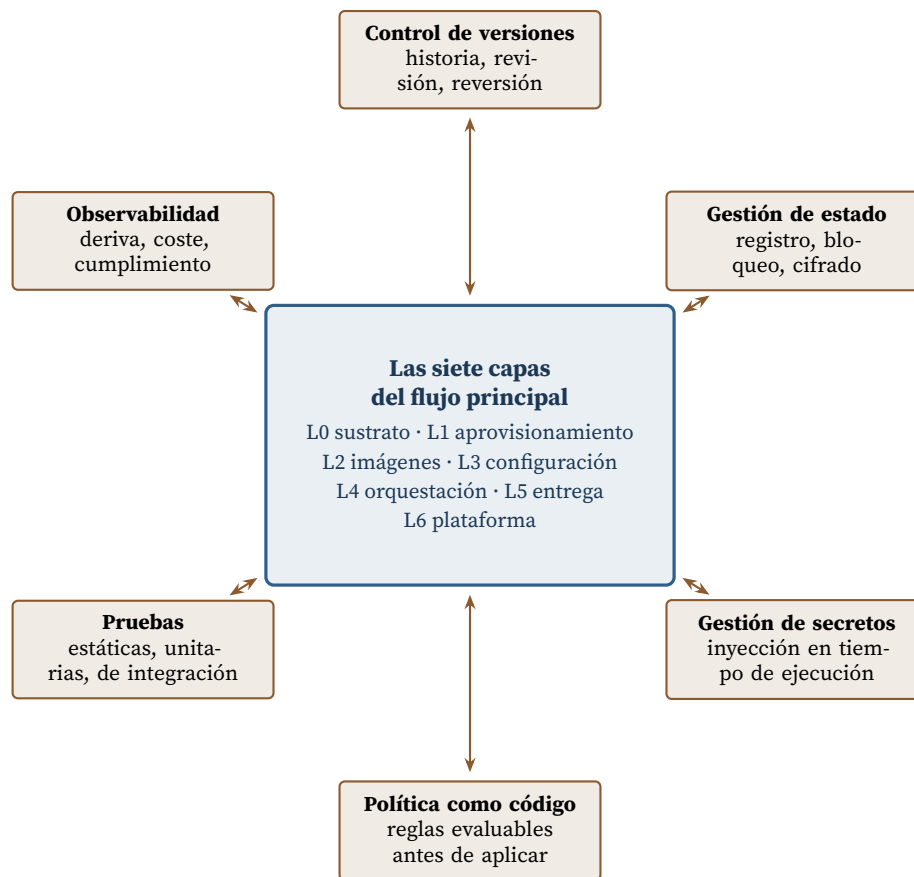


Figura 5: Los seis módulos transversales. Cada uno interactúa con todas las capas del flujo principal, no con una en particular. Elaboración propia.

01. Empuje frente a arrastre

En el modo de **empuje** (*push*), un actor externo, típicamente una canalización de integración, posee las credenciales del entorno y ejecuta el motor contra él. En el modo de **arrastre** (*pull*), un agente residente dentro del entorno consulta el repositorio y se aplica a sí mismo los cambios. La Figura 6 contrasta ambos.

El arrastre tiene dos ventajas estructurales. La primera es de seguridad: las credenciales de producción no viajan a un sistema de integración compartido. La segunda es que el agente, por estar siempre presente, puede conciliar de forma continua y no sólo cuando alguien confirma un cambio, que es la cuarta propiedad exigida en la sección 1. Su coste es que exige un agente por entorno y complica los flujos que necesitan orden entre entornos.

02. Mutable frente a inmutable

En el modelo mutable, un recurso desplegado se modifica en su sitio. En el inmutable, no se modifica nunca: se construye uno nuevo con la configuración deseada y se sustituye. El modelo inmutable elimina por construcción la deriva de configuración y hace que el recurso desplegado sea siempre trazable a un artefacto concreto. A cambio, obliga a que todo estado persistente viva fuera del

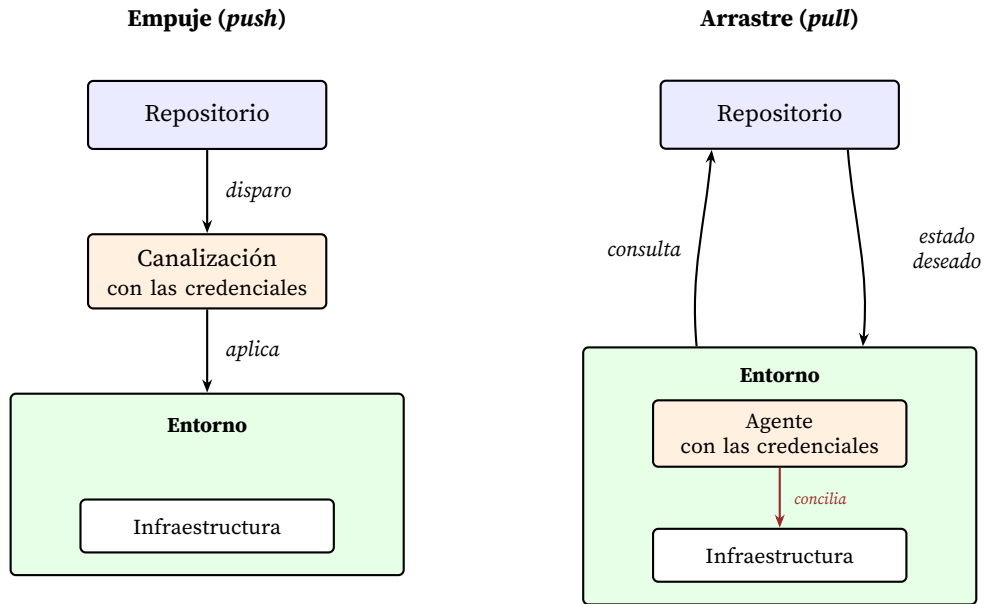


Figura 6: Empuje frente a arrastre. En el arrastre, las credenciales del entorno nunca salen de él, lo que reduce la superficie de ataque y hace del agente el punto natural de conciliación continua. Elaboración propia a partir de los principios de OpenGitOps (s.f.).

recurso reemplazable, lo que es una restricción de diseño exigente.

03. El cuadro de decisión

Las tres decisiones son independientes entre sí, de modo que caben ocho combinaciones. La más frecuente en implantaciones recientes es la de arrastre, inmutable y declarativa, pero ninguna de las tres opciones es obligada. El Cuadro 4 resume las consecuencias de cada una.

Cuadro 4: Tres decisiones de operación y sus consecuencias.

Decisión	Opción A	Opción B
Iniciativa	Empuje: control de orden entre entornos; credenciales fuera del entorno; conciliación sólo bajo disparo.	Arrastre: credenciales confinadas; conciliación continua; un agente por entorno.
Mutabilidad	Mutable: cambios rápidos y baratos; deriva acumulativa; difícil trazar qué versión corre.	Inmutable: sin deriva; reemplazo trazable; obliga a externalizar todo estado persistente.
Expresión	Declarativa: el motor calcula el camino; correcta desde cualquier estado de partida.	Imperativa: control fino del orden; sólo correcta desde el estado de partida previsto.

7. El módulo como unidad de composición

El término «módulo» tiene en IaC un sentido preciso: es la unidad reutilizable que encapsula un conjunto de recursos tras una interfaz de entradas y salidas. Es el mecanismo que permite que una

organización no repita la misma configuración en cada equipo y, sobre todo, que las decisiones correctas queden incorporadas por omisión.

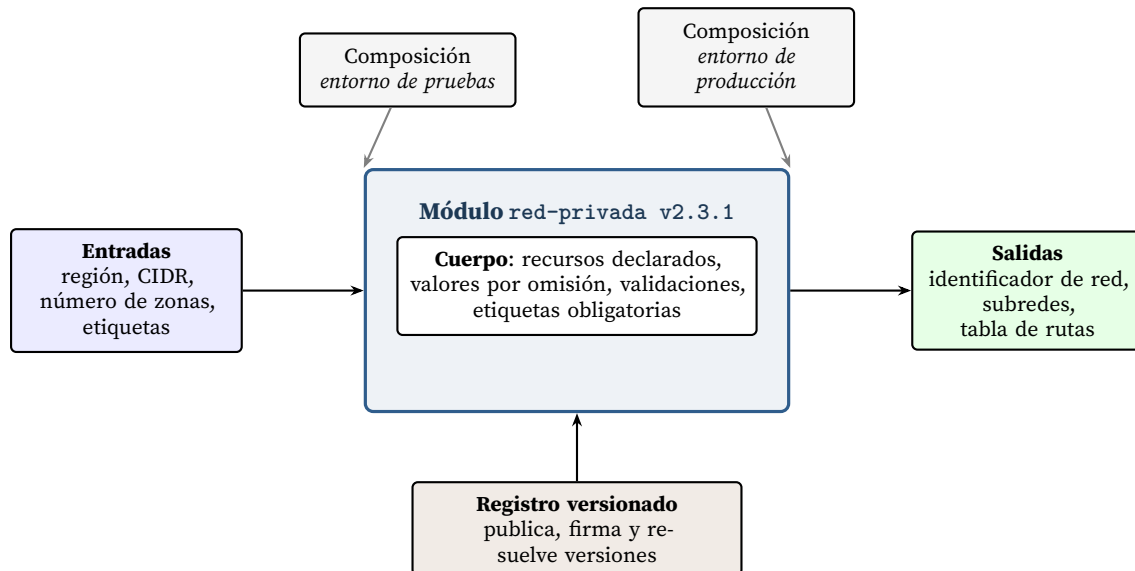


Figura 7: Anatomía de un módulo. Las mismas entradas con valores distintos producen entornos comparables; el registro versionado permite que una corrección se propague de forma controlada. Elaboración propia.

Tres propiedades hacen que un módulo cumpla su función. Debe tener una **interfaz estrecha**: cuantas menos entradas expone, más decisiones fija y más valor aporta. Debe estar **versionado** con semántica explícita, para que quien lo consume elija cuándo adoptar un cambio. Y debe delimitar un **radio de explosión** razonable: un módulo que agrupa demasiados recursos obliga a que cada cambio pequeño planifique sobre un conjunto grande, lo que alarga el ciclo y aumenta el riesgo de cada aplicación.

8. Matriz de estructura de dependencia (DSM)

Esta sección corresponde al elemento 2 del índice de una hoja de ruta tecnológica. Las secciones anteriores han enumerado las piezas del sistema, pero enumerarlas no basta: lo que decide el orden de una implantación, el riesgo de cada paso y la conveniencia de agrupar unos trabajos y separar otros son las dependencias entre ellas. La matriz de estructura de dependencia, o DSM (Eppinger & Browning, 2012; Steward, 1981), es el instrumento que las hace visibles y operables, porque permite reordenar filas y columnas hasta que la propia matriz muestra la secuencia.

Se construyen dos matrices, porque la hoja de ruta necesita responder a dos preguntas distintas. La primera es de arquitectura: qué componente del sistema necesita a qué otro, lo que determina qué se puede diseñar y desplegar por separado. La segunda es de cartera: qué proyecto de la hoja de ruta necesita que otro esté terminado, lo que determina el calendario y el orden de inversión.

01. Convención de lectura

Ambas matrices son cuadradas y llevan los mismos elementos en filas y en columnas, en el mismo orden. Se adopta la convención de entradas por fila:

Una marca en la celda de la fila i y la columna j significa que el elemento i **necesita algo del** elemento j . Se lee, por tanto, recorriendo la fila i de izquierda a derecha: las marcas de esa fila son todo lo que i recibe. Recorriendo en cambio la columna j de arriba abajo se obtiene todo lo que j entrega.

De esa convención se derivan las tres lecturas que se usan después. La **suma de una fila** mide cuántas condiciones previas tiene un elemento, es decir, su dificultad de arranque. La **suma de una columna** mide a cuántos elementos afecta, es decir, el alcance del daño si falla o se retrasa. Y una marca **por encima de la diagonal**, una vez ordenada la matriz, señala una realimentación: un elemento que necesita algo que en el orden propuesto todavía no existe, lo que obliga a iterar.

La diagonal no se marca: ningún elemento se necesita a sí mismo. Las celdas diagonales se somborean únicamente para orientar la vista.

02. La DSM de arquitectura

Los elementos son los trece componentes ya descritos: las siete capas de la Figura 4 y los seis módulos transversales del Cuadro 3. El Cuadro 5 fija los códigos que usa la matriz y, para cada componente, el tipo de dependencia que introduce.

Cuadro 5: Los trece componentes de la DSM de arquitectura y la dependencia que cada uno introduce en el sistema.

Cód.	Componente	Qué necesita de los demás
L0	Sustrato del proveedor	Nada del sistema: es su objeto.
L1	Aprovisionamiento	La interfaz de L0, la especificación versionada, el registro de estado, las credenciales y el veredicto de la política.
L2	Imágenes	Sustrato donde construir, definiciones versionadas y credenciales del registro de artefactos.
L3	Configuración	Máquinas ya creadas por L1 a partir de imágenes de L2.
L4	Orquestación	El agrupamiento aprovisionado por L1 y las imágenes de contenedor de L2.
L5	Entrega	El motor de L1, el destino de L4, el repositorio, las políticas y las pruebas que ejecuta en cada paso.
L6	Plataforma	El catálogo que ejecuta sobre L1, la canalización de L5, las políticas que codifica y las métricas que muestra.
M1	Control de versiones	Nada: es la raíz del sistema.
M2	Gestión de estado	Almacenamiento remoto en L0 y claves de cifrado del módulo de secretos.
M3	Gestión de secretos	El servicio de custodia y cifrado de L0.
M4	Política como código	El plan que produce L1 y el repositorio donde viven las reglas.
M5	Pruebas	Entornos efímeros que crea L1 y el repositorio del que parten.
M6	Observabilidad	Telemetría de L0, ejecuciones de sólo lectura de L1, registros de L5 y el estado custodiado por M2.

La Figura 8 presenta la matriz en el orden natural de exposición, capas primero y módulos después. Es el orden en que se ha descrito el sistema, no el orden en que conviene construirlo, y por eso las marcas aparecen repartidas a ambos lados de la diagonal.

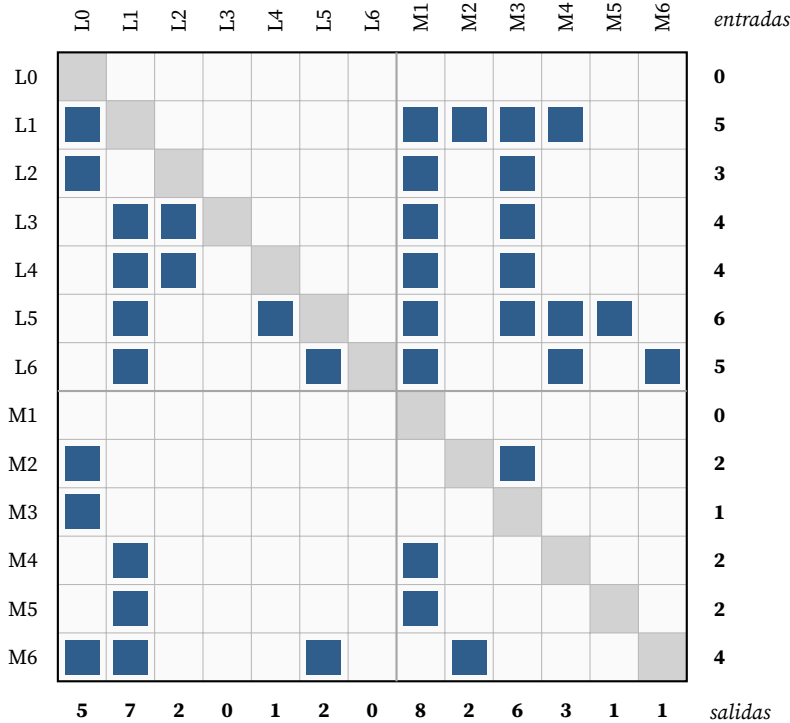


Figura 8: DSM de arquitectura en el orden natural de exposición. Una marca en la fila i y la columna j indica que el componente i necesita algo del componente j . La columna de la derecha suma las entradas de cada fila y la banda inferior suma las salidas de cada columna. Elaboración propia.

03. Qué dice la matriz sin reordenarla

Antes de tocar el orden, las sumas ya dicen tres cosas.

Dos de los tres concentradores de riesgo son módulos transversales, no capas. Las columnas con más salidas son M1, el control de versiones, con ocho; L1, el aprovisionamiento, con siete; y M3, la gestión de secretos, con seis. Es un resultado con consecuencias prácticas: una interrupción del repositorio detiene ocho componentes y una del custodio de secretos detiene seis, mientras que la caída de la capa de configuración, L3, con cero salidas, no detiene a ninguno. La disponibilidad que se exige a cada pieza debería seguir ese orden, y con frecuencia no lo sigue, porque la atención se concentra en el motor, que es sólo el segundo de los tres.

Las capas superiores son las caras de construir, no las de operar. Las filas con más entradas son L5, la entrega, con seis, y L1 y L6 con cinco. Un proyecto que empiece por la interfaz de autoservicio está aceptando cinco condiciones previas simultáneas; empezar por ahí es la causa habitual de que un portal interno quede a medias.

Hay dos elementos sin ninguna entrada, L0 y M1, que son por tanto los únicos puntos de arranque posibles de cualquier secuencia.

04. Particionado: la secuencia de implantación

Reordenar filas y columnas con la misma permutación no cambia la información de la matriz, sólo la presentación. El particionado busca la permutación que lleva el mayor número posible de marcas por debajo de la diagonal, porque una matriz triangular inferior describe un orden en el que cada elemento se construye cuando todo lo que necesita ya existe. La Figura 9 recoge el resultado.

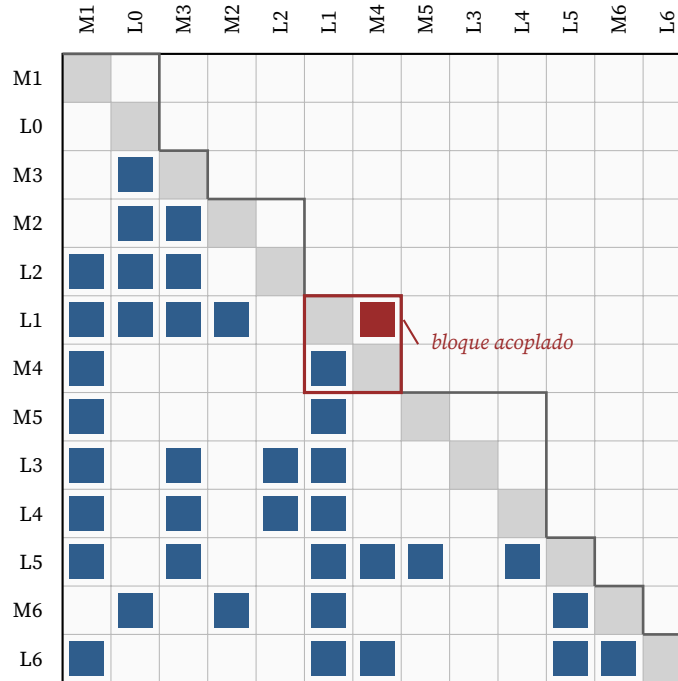


Figura 9: DSM de arquitectura particionada. En el orden M1, L0, M3, M2, L2, L1, M4, M5, L3, L4, L5, M6, L6 todas las dependencias caen por debajo de la diagonal salvo una, la que L1 tiene de M4, marcada en rojo. El escalonado gris separa las olas: los elementos de una misma ola no dependen entre sí y pueden abordarse en paralelo. Elaboración propia.

El resultado es una matriz triangular inferior con una sola excepción, y esa excepción es informativa. El escalonado define ocho olas, y los elementos que comparten ola pueden abordarse en paralelo:

1. **M1 y L0**: el repositorio y la cuenta del proveedor. Sin entradas, son el arranque obligado.
2. **M3**: la gestión de secretos. Aparece la segunda, antes que cualquier capa, porque seis componentes la necesitan y ninguno puede autenticarse sin ella. Ponerla al final, que es lo habitual cuando se trata como un endurecimiento posterior, obliga a rehacer todo lo anterior.
3. **M2 y L2**: la custodia del registro de estado y la construcción de imágenes, que no dependen la una de la otra.
4. **L1 y M4**: el bloque acoplado, del que se habla enseguida.
5. **M5, L3 y L4**: las pruebas, la configuración y la orquestación, los tres apoyados en el motor ya operativo.
6. **L5**: la entrega continua, que necesita el motor, la orquestación, las políticas y las pruebas a la vez.
7. **M6**: la observabilidad, que sólo puede medir lo que ya funciona.
8. **L6**: la interfaz de autoservicio, la última porque envuelve a todo lo demás.

El bloque acoplado y cómo se rompe. L1 necesita de M4 el veredicto de la política antes de aplicar un cambio, y M4 necesita de L1 el plan sobre el que pronunciarse. En una DSM eso es un ciclo, y un ciclo obliga a iterar o a descomponer. La descomposición correcta consiste en partir L1 en dos subprocesos, uno que produce el plan y otro que lo aplica, con la evaluación de política entre ambos. Es exactamente lo que hace el diagrama SD1 del modelo OPM de la sección 9: al separar *Planificación, Validación y Aplicación* en subprocesos distintos, el ciclo desaparece y la secuencia queda lineal. La DSM detecta el acoplamiento y el modelo OPM propone la partición que lo elimina, lo que ilustra por qué los elementos 2 y 3 de la hoja de ruta se sostienen mutuamente.

05. La DSM de la cartera de proyectos

La segunda matriz cambia de objeto. Sus elementos ya no son componentes del sistema sino los ocho proyectos con que la empresa pasa de la plataforma actual a la plataforma de 2029. Tres de ellos se corresponden con los vectores tecnológicos del Cuadro 14, que la sección 11 utiliza para construir el gráfico vectorial; los otros cinco son los habilitadores sin los cuales aquéllos no se sostienen. El Cuadro 6 los enumera.

Cuadro 6: Los ocho proyectos de la cartera, su correspondencia con los vectores tecnológicos del Cuadro 14 y los componentes de la DSM de arquitectura sobre los que actúan. Las filas con la columna *vector* en blanco son proyectos habilitadores, que no desplazan por sí solos ninguna coordenada del gráfico vectorial.

Cód.	Proyecto	Alcance	Vector	Comp.
R1	Cierre de cobertura de codificación	Inventariar los recursos creados a mano e importarlos al motor hasta llevar la cobertura del 55 al 86 por ciento.		L1
R2	Custodia del registro de estado	Estado remoto, cifrado, versionado y con bloqueo de concurrencia.		M2
R3	Catálogo de módulos certificados	Módulos versionados, firmados y publicados en un registro interno, con interfaz estrecha.	P1	L1, L6
R4	Política como código	Reglas de la organización evaluadas sobre el plan antes de aplicarlo.	P1	M4
R5	Malla de pruebas y entornos efímeros	Análisis estático, validación del plan y despliegue efímero verificado.		M5
R6	Sustitución del motor	Migración a la bifurcación abierta con estado gestionado propio.	P2	L1
R7	Interfaz de autoservicio	Portal que expone el catálogo al equipo de producto.	P3	L6
R8	Observabilidad de la plataforma	Deriva, coste, tiempo de ciclo y cumplimiento como series temporales.		M6

Las dependencias entre proyectos no son las mismas que entre componentes, porque un proyecto puede necesitar de otro un resultado parcial y no el componente entero. La Figura 10 distingue por eso dos intensidades: dependencia *necesaria*, cuando el proyecto no puede empezar sin el resultado del otro, y dependencia *conveniente*, cuando puede empezar pero a un coste o un riesgo mayores.

06. Secuenciación de la cartera

Aplicado el mismo particionado, la matriz de la cartera resulta ser completamente triangular inferior: no hay ningún ciclo, de modo que existe una secuencia que no obliga a rehacer nada. La Figura 11 la recoge y la traduce en seis olas.

	R1	R2	R3	R4	R5	R6	R7	R8
R1	■	■						
R2		■						
R3	■	□	■					
R4	□		■	■				
R5		□	■		■			
R6		■	■		■	■		
R7			■	■		□	■	
R8	■	□						■

Figura 10: DSM de la cartera de proyectos en el orden de codificación R1 a R8. Marca llena: el proyecto de la fila no puede empezar sin el resultado del proyecto de la columna. Marca hueca: puede empezar, pero a mayor coste o riesgo. Elaboración propia.

La secuencia resultante es R2; R1; R3 y R8 en paralelo; R4 y R5 en paralelo; R6; y por último R7. Tres consecuencias merecen comentario.

La custodia del estado va primero, y sola. R2 no depende de nada y casi todo depende de él. Es el único proyecto de la cartera que no admite ser pospuesto, porque importar recursos a un registro de estado sin bloqueo ni cifrado, que es lo que haría R1 sin él, multiplica el trabajo por dos: primero se importa y después se migra.

La observabilidad no es el remate, es la ola 3. R8 aparece pronto porque sin él las figuras de mérito del Cuadro 10 no son medibles, y sin medida las posiciones del gráfico vectorial de la sección 11 siguen siendo estimaciones. Situarlo al final, que es la tentación natural, deja la hoja de ruta sin instrumento de verificación justo durante los años en que se ejecuta.

La matriz confirma el orden discutido del catálogo y el motor. El apartado 05 de la sección 11 defiende en prosa que el catálogo de módulos (R3) debe construirse antes que la sustitución del motor (R6), y reconoce que es la decisión más discutible de aquella sección. La DSM lo convierte en una constatación estructural: R6 tiene marca necesaria en las columnas R2, R3 y R5, de modo que en cualquier orden admisible R6 queda por detrás de los tres. Invertir el orden no es una preferencia distinta, es una marca por encima de la diagonal, esto es, una iteración forzada.

07. Riesgos que la matriz hace visibles

Cruzar las dos matrices identifica dónde conviene poner las reservas de esfuerzo, que es el uso que la gestión del portafolio hace de la DSM.

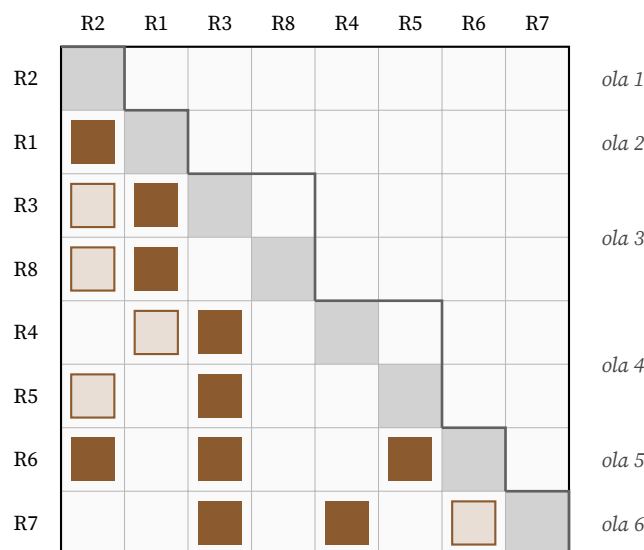


Figura 11: DSM de la cartera particionada. Todas las marcas quedan por debajo de la diagonal: la cartera admite una secuencia sin iteración. R3 y R8 caen en la misma ola porque no dependen entre sí, y lo mismo ocurre con R4 y R5. Elaboración propia.

Cuadro 7: Riesgos derivados de la estructura de dependencias y respuesta prevista.

Origen en la matriz	Riesgo	Respuesta
M1 con ocho salidas	Una indisponibilidad del repositorio detiene ocho componentes; su compromiso los compromete a todos.	Réplica del repositorio y firma obligatoria de confirmaciones.
M3 con seis salidas y ola temprana	Si la gestión de secretos llega tarde, lo construido antes queda con credenciales incrustadas.	Adelantarla a la ola 2 de la arquitectura, antes que cualquier capa intermedia.
Bloque acoplado L1-M4	La política evaluada después de aplicar convierte el incumplimiento en un hecho consumado.	Partir el motor en planificación y aplicación, con la evaluación intercalada, según el diagrama SD1 de la sección 9.
R2 con cinco entradas en su columna	Un retraso en la custodia del estado desplaza toda la cartera.	Ola 1 en solitario, con criterio de terminación explícito antes de abrir R1.
R3 con cuatro entradas en su columna	El catálogo es el cuello de botella de la cartera: R4, R5, R6 y R7 lo esperan.	Publicar el catálogo por incrementos y abrir R4 y R5 contra los primeros módulos, sin esperar al catálogo completo.
L3 y L6 con cero salidas	Ningún componente los espera, luego son los candidatos naturales a recortar si el calendario se comprime.	Registrar el recorte como decisión consciente, no como omisión.

08. Límites de las dos matrices

Tres reservas. La primera es que ambas matrices son **binarias en el tipo**: registran que hay dependencia, no de qué naturaleza es. Una DSM más elaborada distinguiría dependencias de información, de material, de energía y espaciales, y la de este sistema sería casi enteramente de información, lo que resta valor a la distinción.

La segunda es que la DSM de arquitectura describe una **implantación completa desde cero**. Una organización que ya opera parte del sistema, que es el caso de la empresa de referencia, no recorre las olas en orden sino que entra en la secuencia por el punto en que se encuentra; el orden sigue siendo válido para los componentes que faltan.

La tercera es que la matriz de cartera **no incorpora el calendario ni el coste**. Dice qué debe ir antes que qué, no cuánto dura cada cosa ni cuántos equipos caben en paralelo. Convertir estas olas en fechas exige el modelo financiero, el elemento 8 del índice, que este documento declara fuera de alcance.

9. Modelo de objetos y procesos (OPM)

01. Por qué OPM y cómo se ha construido el modelo

Esta sección corresponde al elemento 3 del índice de una hoja de ruta tecnológica. La metodología de objetos y procesos, normalizada como ISO/PAS 19450:2015 y después como ISO 19450:2024 (Dori, 2002, 2016; International Organization for Standardization, 2015, 2024), presenta dos rasgos que la hacen adecuada aquí. El primero es que usa un único tipo de diagrama para representar a la vez estructura y comportamiento, lo que evita el problema habitual de mantener sincronizados un diagrama de bloques y un diagrama de actividad. El segundo es su carácter bimodal: cada diagrama tiene una traducción a lenguaje natural controlado, el OPL, de modo que el modelo se verifica leyéndolo, y un enlace mal puesto se manifiesta como una frase falsa.

Las herramientas autorizadas para este elemento son OPCloud (Enterprise Systems Modeling Laboratory, Technion, s.f.-b) y OPCAT (Enterprise Systems Modeling Laboratory, Technion, s.f.-a), y el modelo está enunciado en el vocabulario cerrado que ambas admiten: cada entidad, cada enlace y cada frase del OPL de los apartados siguientes se corresponden con un elemento de la norma y con ningún otro. Las figuras de esta sección son la composición tipográfica de ese mismo modelo, dibujada con la notación de la norma para que quepa en la caja del documento y pueda referirse desde el texto.

02. Vocabulario y notación empleados

OPM tiene un vocabulario cerrado, y la mayor parte de los errores de modelado consisten en usar un símbolo por otro. El Cuadro 8 fija el que se usa en las tres figuras siguientes.

Dos precisiones sobre el uso de estos símbolos, porque son las dos que con más frecuencia se incumplen. La primera: la precedencia temporal entre subprocesos de una ampliación se expresa **únicamente** por la posición vertical, y no dibujando flechas entre ellos; una flecha entre dos procesos sería un enlace de invocación, que significa otra cosa. La segunda: cuando el diagrama muestra los estados de un objeto y el proceso transforma uno concreto en otro, los enlaces deben partir y llegar **al estado** y no al objeto entero; el enlace de efecto al objeto entero sólo es correcto cuando el cambio de estado no se especifica.

Cuadro 8: Vocabulario de OPM empleado en las Figuras 12, 13 y 14, con la frase OPL que genera cada símbolo.

Elemento	Símbolo	Frase OPL
<i>Entidades</i>		
Objeto	Rectángulo de contorno verde	
Proceso	Elipse de contorno azul	
Estado	Rectángulo redondeado dentro del objeto	O puede estar <i>s1</i> , <i>s2</i> o <i>s3</i> .
Cosa física	Con sombra proyectada	C es física.
Cosa ambiental	Contorno discontinuo	C es ambiental.
<i>Enlaces procedimentales</i>		
Agente	Línea acabada en círculo relleno	O maneja P.
Instrumento	Línea acabada en círculo hueco	P requiere O.
Efecto	Flecha de doble punta	P afecta a O.
Consumo	Flecha del objeto al proceso	P consume O.
Resultado	Flecha del proceso al objeto	P produce O.
Entrada y salida	Flechas entre estados concretos y el proceso	P cambia O de <i>s1</i> a <i>s2</i> .
<i>Enlaces estructurales</i>		
Agregación	Triángulo relleno	A consta de B.
Generalización	Triángulo hueco	B es un A.
Exhibición	Triángulo dentro de triángulo	A exhibe B.
<i>Mecanismos de refinamiento</i>		
Ampliación en detalle	El proceso ampliado contiene a sus subprocesos, ordenados de arriba abajo por precedencia temporal	P se amplía en P1, P2 y P3.
Despliegue	Diagrama aparte con la jerarquía estructural de un objeto	(las de sus enlaces estructurales)

03. Diagrama de sistema (SD)

El diagrama de sistema recoge el proceso principal, quién lo maneja, qué instrumentos necesita y qué objetos transforma o produce. Los estados de la *Infraestructura Desplegada* se declaran aquí, pero las transiciones concretas entre ellos no se especifican todavía: a este nivel el proceso la afecta, sin más, y las transiciones aparecen en la ampliación en detalle. El Cuadro 9 enumera las entidades.

Cuadro 9: Entidades del modelo OPM y diagrama en el que aparecen.

Entidad	Tipo	Papel en el modelo
Aprovisionamiento de Infraestructura	Proceso	Proceso principal; se amplía en detalle en SD1.
Equipo de Plataforma	Objeto físico y ambiental	Agente humano que maneja el proceso.
Especificación de Infraestructura	Objeto informático	El código; instrumento del proceso. Se despliega en SD2. Estados: borrador, validada.
Repositorio Versionado	Objeto informático	Todo del que la especificación es parte (SD2).
Módulo Reutilizable, Configuración de Entorno	Objetos informáticos	Especializaciones de la especificación (SD2).
Versión	Objeto informático	Atributo exhibido por la especificación (SD2).
Motor de IaC	Objeto informático	Instrumento que ejecuta la comparación y la conciliación.
Registro de Estado	Objeto informático	Objeto afectado: el proceso lo lee y lo reescribe en cada aplicación.
Proveedor de Nube	Objeto físico y ambiental	Instrumento: expone la interfaz de programación.
Credenciales	Objeto informático	Instrumento con el que el motor se autentica.
Conjunto de Políticas	Objeto informático	Instrumento de la validación.
Infraestructura Desplegada	Objeto físico	Operando del sistema. Estados: inexistente, conforme, desviada, retirada.
Plan de Cambios	Objeto informático	Resultado de la planificación, instrumento de la aplicación. Estados: pendiente, aprobado.
Registro de Auditoría	Objeto informático	Resultado del proceso.

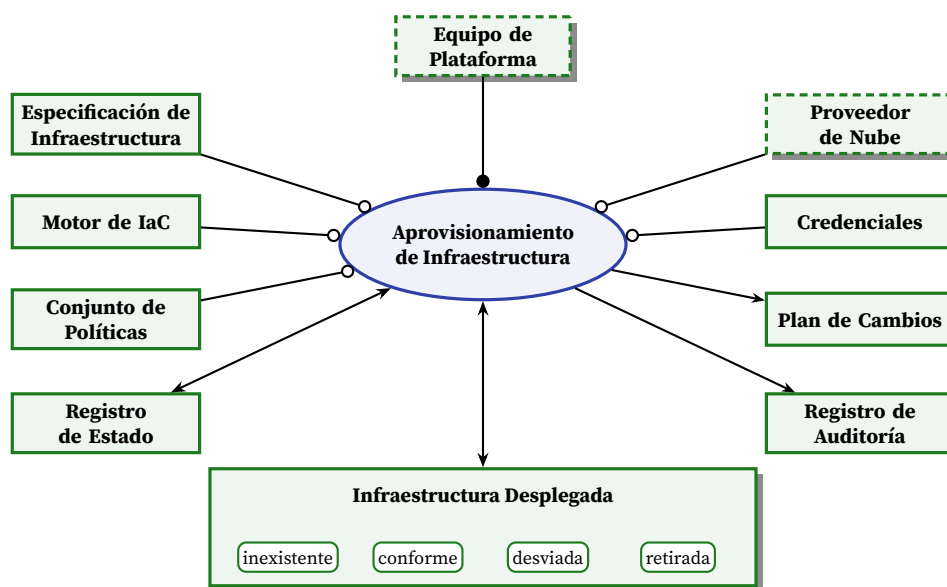


Figura 12: Diagrama de sistema (SD). El proceso principal, su agente, sus cinco instrumentos, el objeto que afecta, los dos que produce y el operando con sus cuatro estados. El *Equipo de Plataforma* y el *Proveedor de Nube* llevan contorno discontinuo por ser ambientales y sombra por ser físicos; la *Infraestructura Desplegada* lleva sombra por ser física. Elaboración propia según Dori (2016) y International Organization for Standardization (2024).

04. El OPL del diagrama de sistema

Las palabras reservadas del lenguaje van en versalitas los nombres de las entidades y en redonda las palabras estructurales. Cada frase se corresponde con un símbolo de la Figura 12 y con ningún otro.

EQUIPO DE PLATAFORMA es físico y ambiental.
 PROVEEDOR DE NUBE es físico y ambiental.
 INFRAESTRUCTURA DESPLEGADA es física.
 INFRAESTRUCTURA DESPLEGADA puede estar inexistente, conforme, desviada o retirada.
 EQUIPO DE PLATAFORMA maneja APROVISIONAMIENTO DE INFRAESTRUCTURA.
 APROVISIONAMIENTO DE INFRAESTRUCTURA requiere ESPECIFICACIÓN DE INFRAESTRUCTURA, MOTOR DE IAC, CONJUNTO DE POLÍTICAS, CREDENCIALES y PROVEEDOR DE NUBE.
 APROVISIONAMIENTO DE INFRAESTRUCTURA afecta a REGISTRO DE ESTADO.
 APROVISIONAMIENTO DE INFRAESTRUCTURA afecta a INFRAESTRUCTURA DESPLEGADA.
 APROVISIONAMIENTO DE INFRAESTRUCTURA produce PLAN DE CAMBIOS y REGISTRO DE AUDITORÍA.

Leído en voz alta, el conjunto es una descripción funcional completa del nivel superior del sistema.

05. Despliegue de la especificación (SD2)

La *Especificación de Infraestructura* tiene una estructura interna que el diagrama de sistema no muestra, porque mezclarla allí saturaría el nivel superior. OPM prevé para eso el despliegue: un diagrama aparte que expone la jerarquía estructural de un objeto sin tocar su comportamiento. La Figura 13 lo recoge con las tres relaciones estructurales de la norma, que son las tres que este objeto necesita.

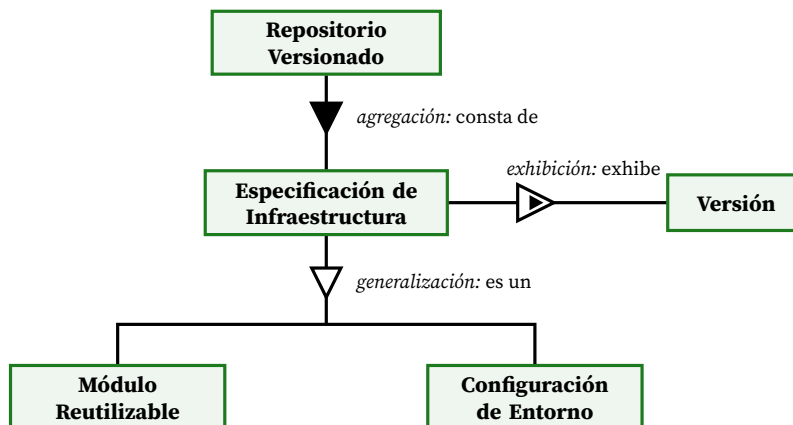


Figura 13: SD2: despliegue del objeto *Especificación de Infraestructura*. Agregación con el repositorio que la contiene, generalización en sus dos especializaciones y exhibición del atributo que la versiona. La base de cada triángulo se apoya en el todo, el general o el exhibidor, y el vértice apunta a la parte, la especialización o el atributo. Elaboración propia.

REPOSITORIO VERSIONADO consta de ESPECIFICACIÓN DE INFRAESTRUCTURA.
 ESPECIFICACIÓN DE INFRAESTRUCTURA exhibe VERSIÓN.
 MÓDULO REUTILIZABLE es una ESPECIFICACIÓN DE INFRAESTRUCTURA.
 CONFIGURACIÓN DE ENTORNO es una ESPECIFICACIÓN DE INFRAESTRUCTURA.

Conviene subrayar la diferencia entre la tercera frase y una que se le parece mucho pero significa otra cosa. *Es un* declara una especialización, esto es, una clase de objeto distinta que hereda las propiedades de la general. *Puede estar* declara un estado, esto es, una situación transitoria del mismo

objeto. Un módulo reutilizable no es un estado de la especificación, es un tipo de especificación; borrador y validada, en cambio, sí son estados.

06. Ampliación en detalle del proceso principal (SD1)

El diagrama SD1 amplía el proceso principal en siete subprocesos. La convención de la norma es que la posición vertical, y sólo ella, denota precedencia temporal: no hay flechas entre subprocesos. Los objetos que ya aparecen en el diagrama de sistema permanecen fuera de la elipse ampliada, y los enlaces cruzan su contorno hasta el subproceso concreto que los usa, que es lo que permite ver en qué punto exacto interviene cada uno. Los estados de la *Especificación de Infraestructura*, del *Plan de Cambios* y de la *Infraestructura Desplegada* se hacen explícitos aquí, y con ellos las transiciones.

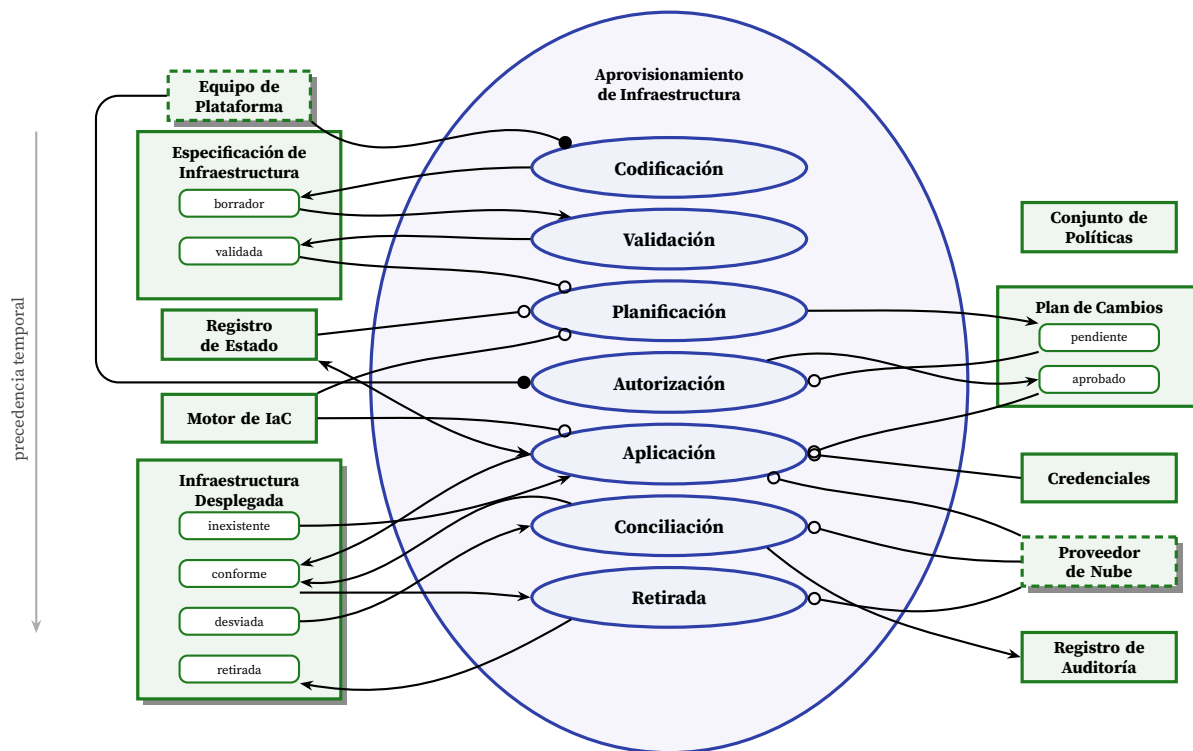


Figura 14: SD1: ampliación en detalle de *Aprovisionamiento de Infraestructura* en sus siete subprocesos. La precedencia temporal la fija la posición vertical, sin flechas entre procesos. Los objetos que ya aparecen en el diagrama de sistema quedan fuera de la elipse y se enlazan con el subproceso que los usa. *Aplicación*, *Conciliación* y *Retirada* actúan sobre el mismo objeto desde estados de partida distintos, que es la formulación precisa de la diferencia entre desplegar, corregir la deriva y dismantelar. Elaboración propia.

07. El OPL de la ampliación

APROVISIONAMIENTO DE INFRAESTRUCTURA se amplía en CODIFICACIÓN, VALIDACIÓN, PLANIFICACIÓN, AUTORIZACIÓN, APLICACIÓN, CONCILIACIÓN y RETIRADA, en ese orden temporal. ESPECIFICACIÓN DE INFRAESTRUCTURA puede estar borrador o validada. PLAN DE CAMBIOS puede estar pendiente o aprobado.

EQUIPO DE PLATAFORMA maneja CODIFICACIÓN y AUTORIZACIÓN.
 CODIFICACIÓN produce ESPECIFICACIÓN DE INFRAESTRUCTURA borrador.
 VALIDACIÓN requiere CONJUNTO DE POLÍTICAS.
 VALIDACIÓN cambia ESPECIFICACIÓN DE INFRAESTRUCTURA de borrador a validada.
 PLANIFICACIÓN requiere MOTOR DE IAC, REGISTRO DE ESTADO y ESPECIFICACIÓN DE INFRAESTRUCTURA validada.
 PLANIFICACIÓN produce PLAN DE CAMBIOS pendiente.
 AUTORIZACIÓN requiere PLAN DE CAMBIOS pendiente.
 AUTORIZACIÓN cambia PLAN DE CAMBIOS de pendiente a aprobado.
 APLICACIÓN requiere MOTOR DE IAC, PLAN DE CAMBIOS aprobado, CREDENCIALES y PROVEEDOR DE NUBE.
 APLICACIÓN cambia INFRAESTRUCTURA DESPLEGADA de inexistente a conforme.
 APLICACIÓN afecta a REGISTRO DE ESTADO.
 CONCILIACIÓN requiere PROVEEDOR DE NUBE.
 CONCILIACIÓN cambia INFRAESTRUCTURA DESPLEGADA de desviada a conforme.
 CONCILIACIÓN produce REGISTRO DE AUDITORÍA.
 RETIRADA requiere PROVEEDOR DE NUBE.
 RETIRADA cambia INFRAESTRUCTURA DESPLEGADA de conforme a retirada.

08. Qué hace explícito el modelo

Merece la pena señalar cuatro cosas que el modelo obliga a decir y que las descripciones en prosa dejan implícitas.

1. REGISTRO DE ESTADO es el único objeto que aparece a la vez como instrumento de la planificación y como objeto afectado por la aplicación. Esa doble condición es exactamente la razón por la que necesita bloqueo de concurrencia, y el modelo la hace visible sin argumentarla.
2. APLICACIÓN, CONCILIACIÓN y RETIRADA cambian el mismo objeto desde estados de partida distintos. Con enlaces de efecto al objeto entero las tres serían indistinguibles; con enlaces de entrada y salida entre estados quedan formalmente separadas, que es la diferencia entre desplegar, corregir la deriva y dismantelar.
3. El equipo humano interviene sólo en dos de los siete subprocesos, la codificación y la autorización. Eso delimita el alcance real de la automatización y, de paso, señala dónde está el cuello de botella de personas.
4. La VALIDACIÓN se sitúa entre la CODIFICACIÓN y la PLANIFICACIÓN, y la AUTORIZACIÓN entre la PLANIFICACIÓN y la APLICACIÓN. Esa partición es la que rompe el bloque acoplado L1-M4 que la Figura 9 detecta en la matriz de dependencias: el veredicto de la política deja de necesitar un motor que a su vez lo necesita a él, porque cada uno actúa sobre un subproceso distinto.

10. Figuras de mérito del sistema

Un sistema de IaC es en sí mismo una tecnología, y como tal admite figuras de mérito que permiten situarlo y seguir su evolución. El Cuadro 10 propone ocho, con su unidad y su procedimiento de medida. Las cuatro primeras coinciden con las métricas de rendimiento de entrega ampliamente utilizadas (Forsgren et al., 2018); las cuatro últimas son específicas de la infraestructura.

El tiempo de reconstrucción es la figura más reveladora de todas, porque verifica de un solo golpe

Cuadro 10: Figuras de mérito propuestas para un sistema de IaC.

Figura de mérito	Unidad	Cómo se mide
Tiempo de ciclo del cambio	horas	Del registro del cambio en el repositorio a su aplicación efectiva en producción.
Frecuencia de aplicación	cambios/semana	Recuento de aplicaciones correctas por entorno.
Tasa de fallo del cambio	%	Aplicaciones que exigen reversión o corrección inmediata sobre el total.
Tiempo de restablecimiento	minutos	De la detección del fallo a la recuperación del servicio.
Cobertura de codificación	%	Recursos gestionados por el motor sobre el total de recursos existentes en el proveedor.
Tasa de deriva	recursos desviados / 1000 · semana	Diferencias detectadas en las ejecuciones de sólo lectura.
Tiempo de reconstrucción	horas	De un entorno vacío a un entorno equivalente al de producción, partiendo sólo del repositorio.
Cumplimiento de política	%	Planes que superan la evaluación de políticas sin excepción manual.

las cuatro propiedades exigidas en la sección 1. Una organización que no puede reconstruir un entorno completo desde el repositorio no tiene un sistema de IaC completo, con independencia de cuántos ficheros declarativos posea.

11. Posicionamiento frente a la competencia

Esta sección corresponde al elemento 6 del índice de una hoja de ruta tecnológica: el posicionamiento de la empresa con respecto a la competencia expresado sobre las figuras de mérito de la tecnología (de Weck, 2022). El instrumento empleado es el gráfico vectorial, que sitúa a cada jugador en un plano cuyos ejes miden la variación de valor percibida por el cliente y la variación de coste soportada por el productor, y que descompone el desplazamiento de cada uno en los vectores tecnológicos concretos que lo producen. Todo el análisis se apoya en las ocho figuras de mérito definidas en el Cuadro 10.

01. Qué se compara y desde qué punto de vista

Un gráfico vectorial obliga a fijar tres cosas antes de dibujar la primera flecha, y ninguna de ellas es evidente cuando la tecnología analizada es una capacidad interna y no un producto de catálogo.

1. **El productor** es la organización de plataforma: el equipo que construye y opera el sistema de infraestructura como código descrito en las secciones anteriores. El coste que soporta es el coste anual de propiedad de ese sistema, no el coste de la infraestructura desplegada con él.
2. **El cliente** es el equipo de producto que consume la plataforma. El valor que percibe no es abstracto: son las cuatro figuras de mérito del Cuadro 10 que se manifiestan en su trabajo diario, a saber, el tiempo de ciclo del cambio, la cobertura de codificación, la tasa de fallo del cambio y el tiempo de reconstrucción. Las otras cuatro son internas del productor y no entran en el eje de valor.
3. **El producto de referencia** es la plataforma tal como está hoy, en 2026: motor de aprovisionamiento comunitario, estado remoto, canalización de integración continua, sin política como

código, sin catálogo certificado y sin interfaz de autoservicio. Ocupa el origen de coordenadas por construcción, y todo el gráfico mide desviaciones respecto a él.

Los jugadores comparados no son las empresas proveedoras entre sí, sino las posiciones que alcanza una organización adoptante al construir su plataforma sobre cada una de las tres familias tecnológicas al horizonte de 2029, es decir, hoy más tres años. Esta distinción importa: la pregunta que responde el gráfico no es cuál herramienta es mejor en abstracto, sino dónde queda situada la empresa según cuál adopte.

02. Los tres competidores considerados

El Cuadro 11 caracteriza a los tres jugadores. Se han elegido porque cubren las tres estrategias de producto que hoy se disputan la capa L1 de la Figura 4: el estándar de hecho multinube (Terraform), la integración vertical de un proveedor único (Bicep) y la apuesta por los lenguajes de propósito general (Pulumi).

Cuadro 11: Caracterización de los tres jugadores en la capa de aprovisionamiento.

	Jugador A: Terraform	Jugador B: Bicep	Jugador C: Pulumi
Titular	HashiCorp, integrada en IBM desde 2025	Microsoft	Pulumi Corporation
Lenguaje	HCL, lenguaje declarativo propio (HashiCorp, s.f.-b)	DSL declarativo que se traduce a plantillas ARM	lenguajes de propósito general: TypeScript, Python, Go, C#
Ámbito	multinube, con un registro de proveedores muy extenso	exclusivamente Azure	multinube, reutiliza los proveedores de Terraform
Estado	fichero de estado explícito, remoto, cifrado y con bloqueo (HashiCorp, s.f.-c)	sin fichero de estado: lo mantiene el gestor de recursos de Azure (Microsoft, s.f.-b)	fichero de estado, en servicio gestionado o autoalojado
Licencia	BUSL desde 2023 (HashiCorp, 2023); servicio gestionado de pago	gratuito, incluido en la suscripción de Azure (Microsoft, s.f.-c)	núcleo abierto; servicio gestionado con tarifa por recurso (Pulumi, s.f.-a)
Política como código	Sentinel u OPA (HashiCorp, s.f.-a; Open Policy Agent, s.f.)	Azure Policy, integrado en el plano de control	CrossGuard
Reutilización	registro de módulos, público y privado	módulos verificados publicados por el propio proveedor (Microsoft, s.f.-a)	componentes que son clases del lenguaje anfitrión
Barrera de adopción	expresividad limitada de HCL y coste de licencia al crecer	dependencia de un proveedor único	exige competencia real de programación en el equipo de plataforma

Junto a los tres jugadores se sitúa la ruta propuesta para la empresa, que no coincide con ninguno de ellos porque combina un motor compatible de código abierto (The Linux Foundation, 2023) con dos inversiones propias, el catálogo de módulos certificados y la interfaz de autoservicio. El gráfico incluye además una quinta posición, la contención de coste, que se evaluó y se descartó, y cuya única función aquí es mostrar que el cuadrante inferior izquierdo del plano existe y es alcanzable.

03. Construcción de las dos coordenadas

Los dos ejes del gráfico no son magnitudes observables sino índices contruidos, y conviene declarar cómo se calculan antes de leer ninguna posición.

Eje de ordenadas: variación de valor para el cliente. Cada una de las cuatro figuras de mérito orientadas al cliente se convierte en una utilidad acotada entre 0 y 100 mediante interpolación lineal entre dos anclas, el peor valor aceptable y el mejor valor alcanzable con la tecnología disponible, al modo de la exploración de espacios de diseño multiatributo (Ross et al., 2004). Para las figuras en las que menos es mejor,

$$u_i = 100 \cdot \frac{x_i^{\text{peor}} - x_i}{x_i^{\text{peor}} - x_i^{\text{mejor}}},$$

y con el numerador invertido para la cobertura de codificación, que es la única en la que más es mejor. Las anclas adoptadas son: tiempo de ciclo, de 48 a 4 horas; cobertura, del 40 al 95 %; tasa de fallo, del 20 al 3 %; tiempo de reconstrucción, de 72 a 6 horas. El valor agregado es la media ponderada

$$V = 0,30 u_{\text{ciclo}} + 0,25 u_{\text{cobertura}} + 0,25 u_{\text{fallo}} + 0,20 u_{\text{reconstrucción}},$$

y la coordenada del gráfico es $\Delta V = V - V_{\text{ref}}$, con $V_{\text{ref}} = 40,3$ puntos para el producto de referencia. Los pesos reflejan una decisión explícita: se prima la velocidad percibida sobre la capacidad de reconstrucción, porque la primera se experimenta cada día y la segunda sólo ante un incidente grave.

Eje de abscisas: variación de coste para el productor. Es un índice de coste anual de propiedad por cada cien recursos gestionados, con la plataforma actual normalizada a 100 puntos. Se descompone en tres sumandos: licencia y servicio gestionado, ingeniería de mantenimiento del propio sistema de plataforma, y formación e incorporación de personas. La coordenada del gráfico es $\Delta C = C - 100$.

Una advertencia sobre el eje de coste: mide el régimen permanente en 2029, no la inversión de transición. El coste de una sola vez de cada migración pertenece al modelo financiero, que es el elemento 8 del índice y queda fuera de esta sección. Un jugador puede parecer barato en el gráfico y exigir un desembolso inicial considerable para llegar a esa posición.

04. Estimación de las figuras de mérito al horizonte 2029

El Cuadro 12 recoge las estimaciones y las utilidades resultantes. Son estimaciones razonadas de ingeniería, no medidas: se apoyan en el comportamiento observado de la plataforma actual y en las propiedades declaradas de cada tecnología, y su función es ordenar posiciones, no predecir cifras.

El Cuadro 13 descompone el otro eje. La diferencia decisiva no está en la licencia sino en la ingeniería de mantenimiento: un catálogo de módulos certificados y una política como código reducen el trabajo recurrente de revisión y corrección lo bastante para compensar el esfuerzo de construirlos y sostenerlos.

La Figura 15 compara los perfiles de utilidad figura por figura, antes de agregarlos. Muestra algo que la coordenada agregada esconde: las cuatro posiciones avanzadas son casi indistinguibles en velocidad y en fiabilidad, y toda su diferencia se concentra en la cobertura de codificación. Bicep no pierde por ser lento, sino por dejar recursos fuera del alcance del motor.

Cuadro 12: Figuras de mérito estimadas al horizonte 2029 y valor agregado resultante. Las cuatro figuras son las orientadas al cliente del Cuadro 10.

Posición	Ciclo (h)	Cobert. (%)	Fallo (%)	Reconstr. (h)	V	ΔV
Producto de referencia (2026)	26	55	14	40	40,3	0,0
A. Terraform gestionado	9	85	6	12	85,8	+45,5
B. Bicep sobre Azure	12	72	8	16	73,7	+33,4
C. Pulumi	7	88	5	10	90,6	+50,3
Ruta propia	8	86	6	11	87,3	+46,9
D. Contención de coste (descartada)	32	44	15	60	23,7	-16,6

Cuadro 13: Descomposición del índice de coste anual de propiedad por cada cien recursos gestionados. La plataforma actual se normaliza a 100 puntos.

Posición	Licencia y servicio	Ingeniería de mantenimiento	Formación	Total	ΔC
Producto de referencia (2026)	8	76	16	100	0
A. Terraform gestionado	52	60	20	132	+32
B. Bicep sobre Azure	0	54	22	76	-24
C. Pulumi	46	62	36	144	+44
Ruta propia	12	66	18	96	-4
D. Contención de coste (descartada)	6	56	12	74	-26

05. Descomposición en vectores tecnológicos

El gráfico vectorial no dibuja las posiciones finales como puntos aislados sino como la suma de los saltos tecnológicos que conducen a ellas. El Cuadro 14 identifica cada vector, la inversión que representa y su contribución a las dos coordenadas.

El orden de los vectores de la ruta propia no es arbitrario, y es la decisión más discutible de toda la sección. El catálogo de módulos se construye *antes* de sustituir el motor, y no después, porque el catálogo es precisamente lo que abarata la sustitución: cuando los equipos de producto consumen módulos y no recursos, el motor queda detrás de una interfaz estable y cambiarlo deja de ser un proyecto de migración para convertirse en un cambio interno del catálogo. Invertir el orden convertiría P2 en el vector más caro y arriesgado de los tres.

06. El gráfico vectorial

La Figura 16 compone los vectores del Cuadro 14 sobre el plano de las dos coordenadas. Se lee así: todas las trayectorias arrancan del origen, que es la plataforma actual; cada flecha es una inversión tecnológica concreta y se encadena a la anterior, de modo que la posición de un jugador es la suma vectorial de sus inversiones y no un punto postulado; el punto grueso del final de cada cadena es la posición en 2029; y la flecha discontinua es la opción que se evaluó y se descartó. Cuanto más arriba, más valor recibe el equipo de producto; cuanto más a la izquierda, menos cuesta sostener la plataforma. El cuadrante superior izquierdo es, por tanto, el favorable.

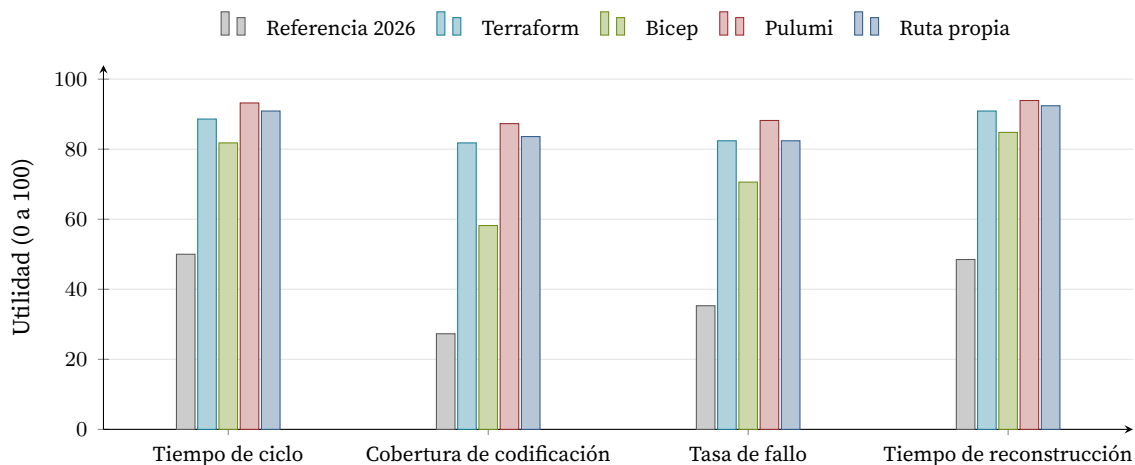


Figura 15: Perfil de utilidad de las cuatro figuras de mérito orientadas al cliente, antes de la agregación ponderada. La única diferencia relevante entre las cuatro posiciones avanzadas está en la cobertura de codificación. Elaboración propia.

07. Lectura del gráfico

El gráfico dice cuatro cosas, y ninguna de ellas es que exista una herramienta mejor que las demás.

Primera: la frontera de eficiencia la forman tres posiciones, no una. Entre las cuatro trayectorias de inversión, Pulumi, la ruta propia y Bicep no se dominan entre sí. Pulumi ofrece el mayor valor, +50,3 puntos, y cuesta +44; Bicep es el más barato, -24, y entrega +33,4; la ruta propia se sitúa en -4 y +46,9. Terraform gestionado, en cambio, sí está dominado: la ruta propia entrega más valor (+46,9 frente a +45,5) por 36 puntos menos de coste. Ésa es la conclusión operativa más limpia de todo el análisis, y explica por qué el vector P2 existe.

Segunda: el valor no lo aporta el motor, lo aportan las capas de encima. En la ruta propia, el vector que sustituye el motor, P2, aporta +1,9 puntos de valor sobre 46,9, es decir, un 4 %. Los otros 45 puntos vienen del catálogo y del autoservicio, que son inversiones propias y no dependen del proveedor elegido. La consecuencia práctica es que la decisión de herramienta, que suele consumir la mayor parte del debate técnico, es la menos determinante de las tres.

Tercera: la Figura 15 explica de dónde sale la distancia. Las cuatro posiciones avanzadas están entre 81 y 93 puntos de utilidad en velocidad y entre 70 y 88 en fiabilidad, márgenes estrechos. La cobertura de codificación, en cambio, va de 58 a 87. Lo que separa a los jugadores no es lo bien que hacen aquello que hacen, sino cuánto de la infraestructura queda dentro de su alcance.

Cuarta: el cuadrante inferior izquierdo es una trampa reconocible. La opción descartada D1 ahorra 26 puntos de coste, dos puntos más que Bicep, y destruye 16,6 puntos de valor. Conviene ser exacto aquí: al ser la más barata de todas, D1 tampoco está dominada, y una regla de eficiencia de Pareto aplicada sin más la conservaría en la frontera. Se descarta por un criterio que el plano no contiene, un suelo de valor aceptable, y esto ilustra el límite del instrumento: el gráfico ordena posiciones, pero no decide cuál es admisible. Reducir el alcance codificado no elimina el trabajo de infraestructura, lo devuelve a la forma manual, donde no se mide y no aparece en el índice de coste de la plataforma porque se ha desplazado a los equipos de producto. El vector es barato en el eje que el gráfico mide y caro en uno que no mide.

Cuadro 14: Descomposición de cada trayectoria en vectores tecnológicos. Las contribuciones son aditivas y suman la posición final de cada jugador.

Vector	Inversión tecnológica que representa	ΔC	ΔV
<i>Jugador A: Terraform</i>			
A1	Ejecución y estado gestionados en el servicio del proveedor, con control de acceso por espacio de trabajo	+24	+30,0
A2	Despliegue multientorno por pilas y política como código con Sentinel	+8	+15,5
<i>Jugador B: Bicep</i>			
B1	Sustitución de las plantillas ARM por el DSL nativo, que suprime el fichero de estado y su gobierno	-20	+14,0
B2	Pilas de despliegue, módulos verificados del proveedor y Azure Policy en el plano de control	-4	+19,4
<i>Jugador C: Pulumi</i>			
C1	Descripción de la infraestructura en un lenguaje de propósito general con pruebas unitarias reales	+32	+26,0
C2	Gestión de entornos y secretos y interfaz programable de automatización (Pulumi, s.f.-b)	+12	+24,3
<i>Ruta propia</i>			
P1	Catálogo de módulos certificados y política como código con OPA (Open Policy Agent, s.f.)	+6	+21,0
P2	Sustitución del motor por la bifurcación abierta y estado gestionado propio (The Linux Foundation, 2023)	-18	+1,9
P3	Interfaz de autoservicio construida sobre el catálogo (The Backstage Authors, s.f.)	+8	+24,0
<i>Opción descartada</i>			
D1	Contención de coste: reducir el alcance codificado al núcleo crítico y devolver el resto a operación manual	-26	-16,6

08. Sensibilidad al supuesto de cartera

Toda la comparación descansa en un supuesto que conviene poner a prueba: que la cartera de infraestructura seguirá concentrada en un solo proveedor de nube. El Cuadro 15 recalcula las cuatro posiciones bajo el escenario contrario, con un 40 % de la carga fuera de Azure, que obliga al jugador B a sostener una segunda cadena de herramientas.

Cuadro 15: Efecto del escenario multinube sobre las posiciones. Sólo el jugador B se desplaza de forma apreciable.

Posición	Proveedor único		Cartera multinube		Desplazamiento	
	ΔC	ΔV	ΔC	ΔV	ΔC	ΔV
A. Terraform gestionado	+32	+45,5	+35	+43,6	+3	-1,9
B. Bicep sobre Azure	-24	+33,4	-4	+10,9	+20	-22,5
C. Pulumi	+44	+50,3	+47	+48,4	+3	-1,9
Ruta propia	-4	+46,9	-1	+45,0	+3	-1,9

El resultado es asimétrico y por eso es útil. Las tres posiciones multinube se mueven tres puntos en coste y dos en valor, un desplazamiento despreciable. El jugador B pierde 22,5 puntos de valor y sube 20 de coste, hasta dejar de ser la opción barata: cae de la frontera de eficiencia al interior del plano, dominado por la ruta propia en ambos ejes. Su ventaja competitiva no es una propiedad de la tecnología sino una propiedad del supuesto de cartera, y desaparece con él.

Esto convierte la elección en una apuesta explícita y no en una preferencia técnica: adoptar Bicep es afirmar que la empresa no abrirá una segunda nube en el horizonte del plan. Si esa afirmación

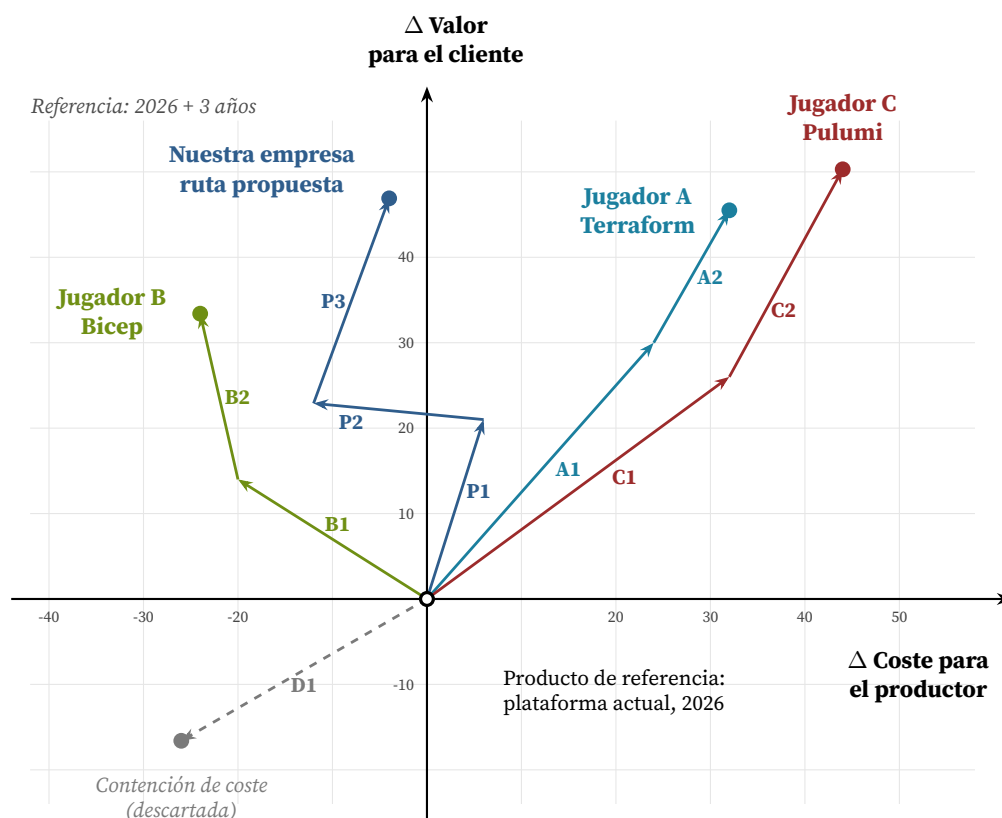


Figura 16: Gráfico vectorial de posicionamiento competitivo al horizonte 2029. El origen es la plataforma actual. Cada flecha es un vector tecnológico del Cuadro 14 y cada punto, la posición que alcanza una organización que adopta esa trayectoria. La rejilla es de diez puntos de índice. Elaboración propia.

no puede sostenerse por escrito ante la dirección, el jugador B queda fuera con independencia de lo que diga el gráfico principal.

09. Límites del análisis

Tres reservas que conviene dejar escritas junto al gráfico, porque un plano con flechas transmite más certidumbre de la que el método admite.

1. **Las coordenadas son estimaciones, no medidas.** Las figuras de mérito del Cuadro 12 proceden del comportamiento observado de la plataforma actual y de las propiedades declaradas de cada tecnología. El propio Cuadro 10 define cómo medirlas: la manera correcta de validar este análisis es instrumentar la referencia y volver a dibujar el gráfico con datos de un trimestre real.
2. **Los pesos del índice de valor son una decisión de gobierno.** Al ponderar el tiempo de ciclo con 0,30 y el de reconstrucción con 0,20 se está escogiendo una postura ante el riesgo. Una organización sujeta a un requisito de recuperación ante desastres invertiría esos pesos, y con ellos el orden de los dos primeros clasificados.
3. **El eje de coste ignora la transición.** Como se advirtió en el apartado 03, mide el régimen

permanente de 2029. La ruta propia exige tres inversiones sucesivas cuyo desembolso inicial no aparece en ninguna coordenada, y esa cifra es la que decide si el plan es financiable. Su lugar es el elemento 8 del índice, el modelo financiero.

12. Conclusiones

1. El funcionamiento de un sistema de infraestructura como código se reduce a un mecanismo único: la conciliación repetida entre un estado deseado, expresado en código versionado, y un estado real, observado a través de la interfaz de un proveedor. Todo lo demás, herramientas, formatos y flujos de trabajo, son variaciones sobre ese mecanismo.
2. La pieza que hace posible el mecanismo es el estado registrado. No es un detalle de implementación sino el elemento que permite reconocer la propiedad de los recursos, detectar destrucciones y encadenar dependencias, y por eso concentra los requisitos de cifrado, versionado y bloqueo.
3. Un sistema completo comprende siete capas, del sustrato del proveedor a la interfaz de autoservicio, y seis módulos transversales. Las capas intermedias compiten por el mismo territorio, de modo que la primera decisión de arquitectura consiste en asignar de forma explícita la propiedad de cada característica a una sola capa.
4. Los módulos transversales, y en particular la política como código y las pruebas, son los que distinguen un sistema que automatiza de uno que además garantiza. Su ausencia no impide desplegar: hace que los errores se desplieguen igual de rápido que los aciertos.
5. El tiempo de reconstrucción de un entorno completo desde el repositorio es la figura de mérito que mejor resume el estado de una implantación, porque verifica de una vez las cuatro propiedades que definen al sistema.
6. La matriz de estructura de dependencia desplaza el orden de implantación que sugiere la intuición. Los concentradores de riesgo no son las capas visibles sino dos módulos transversales, el control de versiones con ocho salidas y la gestión de secretos con seis, y en la secuencia particionada la gestión de secretos aparece en la segunda ola, antes que cualquier capa intermedia. Tratarla como un endurecimiento posterior obliga a rehacer todo lo construido antes.
7. La matriz de la cartera resulta enteramente triangular inferior, de modo que existe una secuencia de los ocho proyectos que no obliga a rehacer ninguno: custodia del estado, cierre de cobertura, catálogo y observabilidad, política y pruebas, sustitución del motor y, en último lugar, interfaz de autoservicio. Convierte además en constatación estructural lo que la sección 11 defiende como criterio: el catálogo antes que el motor no es una preferencia sino la única ordenación que no fuerza una iteración.
8. El gráfico vectorial de la sección 11 desplaza la pregunta habitual. La comparación entre Terraform, Bicep y Pulumi ocupa la mayor parte del debate técnico y explica menos de la décima parte del valor que se puede ganar: en la trayectoria propuesta, el vector que sustituye el motor aporta 1,9 de los 46,9 puntos de mejora, y los otros 45 proceden del catálogo de módulos certificados y de la interfaz de autoservicio, que son inversiones propias e independientes del proveedor elegido.
9. Lo que separa a los jugadores en el plano de las figuras de mérito no es la velocidad ni la fiabilidad, magnitudes en las que las cuatro posiciones avanzadas quedan a pocos puntos entre sí, sino la cobertura de codificación, esto es, cuánta infraestructura queda dentro del alcance del motor. La ventaja de coste del jugador B no es una propiedad de su tecnología

sino del supuesto de proveedor único: bajo una cartera multinube pierde 22,5 puntos de valor y abandona la frontera de eficiencia.

10. El modelado en OPM aporta algo que la descripción en prosa no aporta: obliga a declarar el tipo de cada relación. Al hacerlo aparecen con claridad la doble condición del registro de estado, la diferencia formal entre aplicar, conciliar y retirar, que sólo se sostiene si los enlaces parten de estados y no del objeto entero, y el alcance exacto de la intervención humana, que en este modelo se limita a dos de los siete subprocesos.
11. Los elementos 2 y 3 del índice se sostienen mutuamente, y ese es el hallazgo metodológico del trabajo. La matriz de dependencias detecta un único bloque acoplado, el que forman el motor de aprovisionamiento y la política como código, y no dispone de instrumentos para romperlo; la ampliación en detalle del modelo OPM lo rompe al partir el proceso en planificación, validación y aplicación. La matriz encuentra el problema y el modelo aporta la partición que lo resuelve.

Referencias bibliográficas

- Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5), 50-57. <https://doi.org/10.1145/2890784>
- de Weck, O. L. (2022). *Technology Roadmapping and Development: A Quantitative Approach to the Management of Technology*. Springer. <https://doi.org/10.1007/978-3-030-88346-1>
- Dori, D. (2002). *Object-Process Methodology: A Holistic Systems Paradigm*. Springer. <https://doi.org/10.1007/978-3-642-56209-9>
- Dori, D. (2016). *Model-Based Systems Engineering with OPM and SysML*. Springer. <https://doi.org/10.1007/978-1-4939-3295-5>
- Enterprise Systems Modeling Laboratory, Technion. (s.f.-a). *OPCAT: Object-Process CASE Tool*. Consultado el 7 de septiembre de 2026, desde <https://esml.iem.technion.ac.il/opcat-installation-2/>
- Enterprise Systems Modeling Laboratory, Technion. (s.f.-b). *OPCloud Sandbox*. Consultado el 27 de agosto de 2026, desde <https://opcloud-sandbox.web.app/>
- Eppinger, S. D., & Browning, T. R. (2012). *Design Structure Matrix Methods and Applications*. MIT Press.
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press.
- HashiCorp. (2023, 10 de agosto). *HashiCorp Adopts Business Source License*. Consultado el 1 de septiembre de 2026, desde <https://www.hashicorp.com/en/blog/hashicorp-adopts-business-source-license>
- HashiCorp. (s.f.-a). *Sentinel Documentation*. Consultado el 1 de septiembre de 2026, desde <https://developer.hashicorp.com/sentinel/docs>
- HashiCorp. (s.f.-b). *Terraform Documentation*. Consultado el 1 de septiembre de 2026, desde <https://developer.hashicorp.com/terraform>
- HashiCorp. (s.f.-c). *Terraform State: Purpose*. Consultado el 27 de agosto de 2026, desde <https://developer.hashicorp.com/terraform/language/state/purpose>
- Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
- International Organization for Standardization. (2015, 15 de diciembre). *Automation Systems and Integration: Object-Process Methodology* (Especificación disponible al público ISO/PAS 19450:2015).

- ISO. Ginebra. Consultado el 27 de agosto de 2026, desde <https://www.iso.org/standard/62274.html>
- International Organization for Standardization. (2024, enero). *Automation Systems and Integration: Object-Process Methodology* (Norma internacional ISO 19450:2024). ISO. Ginebra. Consultado el 27 de agosto de 2026, desde <https://www.iso.org/standard/84612.html>
- Microsoft. (s.f.-a). *Azure Verified Modules*. Consultado el 1 de septiembre de 2026, desde <https://azure.github.io/Azure-Verified-Modules/>
- Microsoft. (s.f.-b). *Deployment Stacks*. Consultado el 1 de septiembre de 2026, desde <https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/deployment-stacks>
- Microsoft. (s.f.-c). *What Is Bicep?* Consultado el 1 de septiembre de 2026, desde <https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/overview>
- Morris, K. (2021). *Infrastructure as Code: Dynamic Systems for the Cloud Age* (2.^a ed.). O'Reilly Media.
- Open Policy Agent. (s.f.). *Open Policy Agent Documentation*. Cloud Native Computing Foundation. Consultado el 27 de agosto de 2026, desde <https://www.openpolicyagent.org/docs/latest/>
- OpenGitOps. (s.f.). *GitOps Principles v1.0.0*. Cloud Native Computing Foundation. Consultado el 27 de agosto de 2026, desde <https://github.com/open-gitops/documents/blob/main/PRINCIPLES.md>
- Pulumi. (s.f.-a). *Pulumi Documentation*. Consultado el 1 de septiembre de 2026, desde <https://www.pulumi.com/docs/>
- Pulumi. (s.f.-b). *Pulumi ESC: Environments, Secrets and Configuration*. Consultado el 1 de septiembre de 2026, desde <https://www.pulumi.com/docs/esc/>
- Rahman, A., Mahdavi-Hezaveh, R., & Williams, L. (2019). A Systematic Mapping Study of Infrastructure as Code Research. *Information and Software Technology*, 108, 65-77. <https://doi.org/10.1016/j.infsof.2018.12.004>
- Rahman, A., Parnin, C., & Williams, L. (2019). The Seven Sins: Security Smells in Infrastructure as Code Scripts. *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, 164-175. <https://doi.org/10.1109/ICSE.2019.00033>
- Red Hat. (s.f.). *Ansible Documentation: How Ansible Works*. Consultado el 27 de agosto de 2026, desde https://docs.ansible.com/ansible/latest/getting_started/index.html
- Ross, A. M., Hastings, D. E., Warmkessel, J. M., & Diller, N. P. (2004). Multi-Attribute Tradespace Exploration as Front End for Effective Space System Design. *Journal of Spacecraft and Rockets*, 41(1), 20-28. <https://doi.org/10.2514/1.9204>
- Steward, D. V. (1981). The Design Structure System: A Method for Managing the Design of Complex Systems. *IEEE Transactions on Engineering Management*, EM-28(3), 71-74.
- The Backstage Authors. (s.f.). *What Is Backstage?* Cloud Native Computing Foundation. Consultado el 1 de septiembre de 2026, desde <https://backstage.io/docs/overview/what-is-backstage>
- The Kubernetes Authors. (s.f.). *Kubernetes Documentation: Controllers*. Consultado el 27 de agosto de 2026, desde <https://kubernetes.io/docs/concepts/architecture/controller/>
- The Linux Foundation. (2023, 20 de septiembre). *Linux Foundation Launches OpenTofu: A New Open Source Alternative to Terraform*. Consultado el 27 de agosto de 2026, desde <https://www.linuxfoundation.org/press/announcing-opentofu>